

Verified Verification: A Lean 4 Proof That the Metamath Checker Is Correct

Zarathustra Goertzel (supporting author and human supervisor) Oruži**

September 2026

Abstract

We present a formal verification, carried out entirely in Lean 4 with the Batteries library (no Mathlib), that the Metamath proof checker `checkBytes` correctly validates mathematical proofs. AI assistance: Codex, ChatGPT Pro, and Claude Code (under human supervision). The central result is completeness and soundness for source text, against Mario Carneiro’s declarative semantics: after any prefix that the parser reads without error and that ends between statements with no token pending, in a mode without duplicate `$f` statements, the statement a `$p` claim would store is provable from the assertions read so far *if and only if* the parser reads a rendered continuation (fresh dummy-variable `$v`, `$f` and `$d` statements and one normal proof) and stores exactly that statement with a complete proof. The same holds for the complete file checked by `checkBytes`, and through the entry point `check` for a root file that reads as those bytes. A second biconditional relates proof runs in a parsed database to provability under the Metamath specification (`Spec.Provable`) at the level of expressions. Under a strict configuration (`IsSound`), we additionally prove *prefix provenance*: every accepted theorem is provable using only earlier theorems, ruling out circular reasoning. The development comprises about 82,000 lines of Lean across 103 files, with zero `sorry`s and no axioms beyond Lean’s standard ones. All 65 parser error codes are certified to agree with their recorded evidence and specification clause. The theorems concern source text as the parser reads it (§6).

Contents

1	Introduction	3
2	Metamath Specification Anchors	4
2.1	Core Concepts	4
2.2	Two Formalizations of Provability	4
3	The Main Contract	5
3.1	Completeness for Source Text	5
3.2	Proof Runs in a Parsed Database	6
3.3	Semantic Bridge	7
3.4	Compressed Proofs Reduce to Normal Proofs	7
4	The Generic Spec Theorem	8

“Oruži” denotes AI-assisted collaboration (Codex, ChatGPT Pro, Claude Code; mostly 5. and 4.* series).

5	Prefix Provenance	9
5.1	The Circularity Problem	9
5.2	The IsSound Predicate	10
5.3	The Prefix Provenance Theorem	10
6	What Is Not Claimed	11
7	Reproducing Mechanically	12
7.1	Build Instructions	12
7.2	Verifying Sorry-Freedom	12
7.3	Reviewing Strategy	13
8	Architecture and Proof Strategy	13
8.1	Layer Diagram	13
8.2	Theorem Chain	14
8.3	Key Proof Techniques	14
8.4	Statistics	15
9	Conclusion	15
A	Key Theorem Listings	16
A.1	Spec.ProofValid (Spec/Operational.lean)	16
A.2	statementProvable_iff_exists_extendedFrame (Spec/Completeness.lean)	16
A.3	proofChecker_normal_iff_frameDerivable_in_parsedDB (ParserEquivalence.lean)	17
A.4	checkBytes_parseErrorCode?_fullyCertified (ErrorCodeSemantics.lean)	17
A.5	ModeConfig.IsSound and sound (Verify.lean, Verify/Conformance.lean)	17
B	File Inventory	17
C	Unit Tests and Spec Alignment	18
C.1	§4.1.1 Preliminaries	18
C.2	§4.1.2 Preprocessing	22
C.3	§4.2.1 User-Defined Tokens	26
C.4	§4.2.2 Constants and Variables	27
C.5	§4.2.3 The \$c and \$v Declaration Statements	28
C.6	§4.2.4 The \$d Statement	29
C.7	§4.2.5 The \$f and \$e Statements	30
C.8	§4.2.6 Assertions (\$a and \$p Statements)	33
C.9	§4.2.8 Scoping Statements (\${ and \$})	35
C.10	§4.4.5 Compressed Proof Format	36
C.11	§4.4.6 Unknown Proofs or Subproofs	41
C.12	Appendix E: Metamath Language EBNF	41
C.13	Proof Verification Failures (§4.1.4)	42
C.14	Active Hypotheses (§4.1.4)	44
C.15	Disjoint Variable Violations (§4.1.4)	44
C.16	Valid Databases (Positive Tests)	46
C.17	Miscellaneous	55
C.18	Large Test Files (Summary Only)	56

1 Introduction

Metamath [1] is a minimalist proof language used by a community of formalists to develop large databases of checked theorems, including `set.mm` (over 40,000 theorems in ZFC set theory). Its simplicity—constants, variables, substitution, disjoint-variable constraints, and a stack-based proof checker—makes it an attractive target for formal verification.

Several implementations of the Metamath proof checker exist, among them `metamath.exe` (C), `mmj2` (Java), `metamath-knife` (Rust), `mmverify.py` (Python), and a Lean 4 verifier by Mario Carneiro, from which the checker studied here, `checkBytes` in `Verify.lean`, is derived. They are tested extensively, but none of them is *formally verified*: proved correct against a mathematical specification.

In this paper we close that gap. We prove, in Lean 4, that `checkBytes` is both *sound* (if it accepts a proof, the theorem is genuinely provable) and *complete* (if the statement a claim stores is provable, the checker accepts source text that declares the dummy variables a proof needs and proves it). These two directions are combined into biconditionals for source text and for complete files (§3).

Key results.

1. **Completeness for source text**: in a mode without duplicate `$f` statements, a claim’s stored statement is provable in Mario’s semantics from the assertions read so far iff the parser accepts a rendered continuation, or the completed file, storing exactly that statement with a complete proof (`statementProvable_iff_sourceAccepts`, `statementProvable_iff_fileAccepts`); `check_accepts_of_statementProvable` carries this to `check`.
2. **Proof runs** (`proofChecker_normal_acceptance_iff_specProvable_in_parsedDB`): from raw `ByteArray` to `Spec.Provable` at the level of expressions, in the default configuration.
3. **Frame-derivability bridge** (`proofChecker_normal_iff_frameDerivable_in_parsedDB`): the same biconditional bridged to derivability by Mario’s rules in the frame (`FrameDerivable`) over total database extraction.
4. **Completeness for Mario’s semantics** (`statementProvable_iff_exists_extendedFrame`): a stored statement is provable in Mario’s statement-level semantics iff some extended frame of it has a Metamath proof.
5. **Prefix provenance** (`checkBytes_finalState_finishProofEvent_certified`): every accepted `$p` theorem is provable in the *pre-insertion* database, under a strict configuration.
6. **Compressed proofs reduce to normal proofs** (`ProofReachableZ_iff_NormalProofReachable` $\hookrightarrow$): every compressed or Z-compressed proof run yields a normal one, at the database level.
7. **Error code certification** (`checkBytes_parseErrorCode?_fullyCertified`): all 65 error codes agree with their evidence payload.

Trust boundary. The theorems are checked by Lean’s kernel. The core semantic theorems are stated over `checkBytes` on `ByteArray` input. The default IO checker is `check`, which performs single-pass include handling with bounded depth (`ModeConfig.maxIncludeDepth`) around a pure parser-step core. For a single root file, `check` returns what `checkBytes` returns on the file’s bytes, when the include depth limit is positive, the path resolves, the read returns those bytes, and the

pure run raises no include request (`RootFileCheck.lean`). Source text with include directives is not covered by the completeness theorems. The filesystem calls (`realPath`, `readBinFile`) are trusted, and running the compiled checker also relies on the Lean compiler and runtime.

Organization. §2 maps the Metamath specification to Lean types. §3 presents the main contract theorems. §4 describes the generic spec-level bridge. §5 covers prefix provenance. §6 documents what is *not* claimed. §7 gives reproduction instructions. §8 describes the architecture and proof strategy. §9 concludes. Appendix A contains key theorem listings, Appendix B provides a file inventory, and Appendix C lists all unit tests grouped by specification rule.

2 Metamath Specification Anchors

The Metamath language is defined in Chapter 4 of the Metamath book [1]. We briefly recap the core concepts and their Lean formalizations.

2.1 Core Concepts

A Metamath database declares *constants* (fixed symbols like `wff`, `|-`, `()`) and *variables* (placeholders that can be substituted). An *expression* is a sequence of math symbols (§4.2.2); in a hypothesis or an assertion it begins with a constant, its *typecode*. Both formalizations below represent such an expression as a typecode together with the list of the remaining symbols. A *substitution* maps variables to expressions.

Disjoint variable (DV) constraints (`$d` statements) restrict which variables may share substituted variables—this is the mechanism that prevents unsound capture.

A *frame* collects the mandatory hypotheses (floating `$f` and essential `$e`) and DV constraints for an assertion. Table 1 shows the correspondence between specification sections and Lean types.

Table 1: Sections of the Metamath book mapped to the types of `DeclarativeSpec.lean`.

Section	Topic	Lean type
§4.2.2	Constants, variables, expressions	CN, VR, Sym, Expr
§4.1.4	Substitution	Expr.subst
§4.2.4	Disjoint variable restrictions	DJ, DJ.subst
§4.2.5	Floating and essential hypotheses	Formula, VR.vhyp
§4.2.6	Assertions (<code>\$a</code> , <code>\$p</code>)	Statement
§4.2.7	Frames	Context
§4.1.4	Proof verification	Provable

2.2 Two Formalizations of Provability

We maintain two independent formalizations of what it means for a theorem to be provable:

Operational (`Spec.Provable`). Defined in `Spec/Operational.lean`, this models the Metamath proof checker as a stack machine. A proof is a sequence of steps; each step either pushes a hypothesis or applies an assertion (popping arguments and pushing the conclusion). A formula is provable if there exists a step sequence that produces a singleton stack containing it.

```

1 def Provable (G : Database) (fr : Frame) (e : Expr) : Prop :=
2   exists (steps : List ProofStep) (finalStack : List Expr),
3     ProofValid G fr finalStack steps
4     and finalStack = [e]

```

Listing 1: Operational provability (`Spec/Operational.lean`).

The `ProofValid` inductive relation (Listing A.1 in Appendix A) defines the step-by-step stack machine execution, with constructors for essential hypotheses, floating hypotheses, and axiom/theorem application (including DV checking and type preservation).

Declarative (`Declarative.Provable`). Defined in `DeclarativeSpec.lean`, after Mario Carneiro’s Lean specification of Metamath, this is a “big-step” inductive with three constructors: use a hypothesis directly, type any variable, or apply an axiom with a substitution that satisfies its DV constraints and whose hypothesis instances and variable typings are all provable.

```

1 inductive Provable (axs : Statement -> Prop) (G : Context)
2   : Formula -> Prop
3   | hyp (h) : h in G.hyps -> Provable axs G h
4   | var (v:VR) : Provable axs G v
5   | ax (s) {ax} : axs ax -> ax.ctx.dj.subst s G.dj ->
6     (forall h in ax.ctx.hyps, Provable axs G (h.subst s)) ->
7     (forall v in ax.vars, Provable axs G (v.type, s v)) ->
8     Provable axs G (ax.fmla.subst s)

```

Listing 2: Declarative provability (`DeclarativeSpec.lean`).

The typing premise of the `ax` rule ranges over `ax.vars`, the variables of the applied statement, as in clause 2(a) of the closure in Appendix C.2.5 of the book. Mario Carneiro’s original rule asks for a typing of every variable; `Spec/DeclarativeOriginal.lean` keeps that rule and proves that the two rules make the same statements provable over trimmed axiom sets, which include every set of assertions a parsed database yields (`DeclarativeOriginal.statementProvable_iff`, `dbToAxioms_trimmed`).

The operational and declarative formulations are proven equivalent in §4.

3 The Main Contract

The central theorem of the project connects the *actual Lean implementation* (the parser of `Verify.lean`) to Mario Carneiro’s declarative semantics (`DeclarativeSpec.lean`) at the level of source text.

3.1 Completeness for Source Text

Let the parser read the text `arr` from the start without error, in a mode without duplicate `$f` statements, ending between statements with no token pending. Take a claim `f` with a constant head and declared symbols under a fresh label token, whose frame trims to `frImpl` with stored statement `fr`.

```

1 theorem statementProvable_iff_sourceAccepts ... :
2   (statementOfFrame fr (toExpr f)).Provable
3   (dbToAxioms (toDatabaseTotal (afterSource config arr).db))
4   <->

```

```

5   exists ds proof, Admissible (afterSource config arr) ds label f proof
6     and SourceAccepts config arr
7       (render (afterSource config arr) ds label f proof) label f frImpl

```

Listing 3: Completeness for source text (SourceCompleteness.lean).

`render` emits `$v` and `$f` statements for the dummy variables `ds`, the `$d` pairs the parser stores for them, and one `$p` statement with the normal proof `proof`. `Admissible` requires fresh names, legal label and math-symbol tokens, and proof steps that are label tokens (so no `?` step and no injected commands). `SourceAccepts` says that the parser reads the text after `arr` without error, ends between statements, stores `label` as exactly `f` with the frame `frImpl`, and records no new incomplete proof. Companion theorems show that the declarations leave the assertion database and the claim’s trimmed frame unchanged, and that the continuation adds exactly the one assertion. `statementProvable_iff_fileAccepts` states the same for the complete file, closed by one `$}` per open block and checked by `checkBytes` with its post-checks, and `check_accepts_of_statementProvable` with `statementProvable_of_check` carries it to check for a root file that resolves and reads as exactly those bytes.

The axioms on the left are the assertions read so far: the `$a` statements and the earlier `$p` statements, including, in modes that accept `?` steps, theorems whose proofs are incomplete. Naming `fr` is no restriction: a frame that trims at such a prefix always has a spec frame (`exists_toFrame_of_trim`). For Mario Carneiro’s original `ax` rule, the same is stated by `statementProvable_iff_sourceAccepts_originalRule` and `statementProvable_iff_fileAccepts_originalRule`.

3.2 Proof Runs in a Parsed Database

```

1  theorem proofChecker_normal_acceptance_iff_specProvable_in_parsedDB
2    (bytes : ByteArray)
3    (label : String)
4    (f : Verify.Formula)
5    (h_success : (Verify.checkBytes bytes).error? = none) :
6    (exists (proof : Array String) (pr_final : Verify.ProofState)
7      (f' : Verify.Formula),
8      proof.foldlM (fun pr step =>
9        Verify.DB.stepNormal (Verify.checkBytes bytes) pr step)
10      <..., label, f, (Verify.checkBytes bytes).frame,
11        #[], #[], .normal> =
12        Except.ok pr_final
13      and pr_final.stack.size = 1
14      and pr_final.stack[0]? = some f'
15      and toExpr f' = toExpr f)
16    <=>
17    (exists (G : Spec.Database) (fr : Spec.Frame),
18      toDatabase (Verify.checkBytes bytes) = some G
19      and toFrame (Verify.checkBytes bytes)
20        (Verify.checkBytes bytes).frame = some fr
21      and Spec.Provable G fr (toExpr f))

```

Listing 4: Proof runs (KernelCorrectness.lean).

Reading the theorem. The left-hand side says: there exists a proof array such that folding `stepNormal` over it (starting from an initial proof state with the given label and formula) succeeds,

producing a singleton stack whose top element has the same `toExpr` image as `f`. The right-hand side says: the database and frame extracted from the parser state witness `Spec.Provable` for the `toExpr` image of `f`.

The only hypothesis is `h_success`: the parser completed without error, in the default configuration (`checkBytes bytes`). It concerns proof runs, not acceptance: `finishProof` compares the final stack with the claim literally, while `toExpr` maps the empty formula and `ERROR` to the same expression. `verify_impl_complete_exact` gives the literal final stack.

3.3 Semantic Bridge

The biconditional is further bridged to Carneiro’s declarative semantics:

```

1  theorem proofChecker_normal_iff_frameDerivable_in_parsedDB
2    (bytes : ByteArray)
3    (label : String)
4    (f : Verify.Formula)
5    (h_success : (Verify.checkBytes bytes).error? = none) :
6    (exists (proof : Array String) (pr_final : Verify.ProofState)
7      (f' : Verify.Formula),
8      proof.foldlM (...) = Except.ok pr_final
9      and pr_final.stack.size = 1
10     and pr_final.stack[0]? = some f'
11     and toExpr f' = toExpr f)
12  <->
13  (exists (fr : Spec.Frame),
14    toFrame (Verify.checkBytes bytes)
15      (Verify.checkBytes bytes).frame = some fr
16    and FrameDerivable
17      (toDatabaseTotal (Verify.checkBytes bytes))
18      fr
19      (exprToFormula (varMapOfFrame fr) (toExpr f)))

```

Listing 5: Semantic bridge (`ParserEquivalence.lean`).

This is the canonical completeness interface at one fixed frame. Completeness with respect to Mario’s own semantics is stated for stored statements and extended frames (`statementProvable_iff_exists_extendedFrame, Spec/Completeness.lean`).

3.4 Compressed Proofs Reduce to Normal Proofs

Metamath supports two proof formats: *normal* (explicit label sequence) and *compressed* (a compact encoding). We prove that every compressed run yields a normal run, so reachability in any format is normal reachability at the database level:

```

1  theorem
2    ProofReachableZ_iff_NormalProofReachable
3    (db : DB) (label : String) (fmla : Formula) (stack : Array Formula)
4    (h_wf : WellFormedDB db)
5    (h_size : stack.size = 1) (h_fmla : stack[0]? = some fmla) :
6    ProofReachableZ db label fmla stack
7    <-> NormalProofReachable db label fmla stack

```

Listing 6: Compressed-to-normal reduction (`ParserAnyFormatEquivalence.lean`).

A reviewer therefore needs to understand only the normal format to trust accepted compressed proofs. We do not construct compressed proofs from normal ones.

4 The Generic Spec Theorem

Underneath the parser-level biconditionals lies a specification-level equivalence between our operational verifier and Mario’s declarative formulation. This section presents the split as soundness + completeness, which is the natural structure of the proof.

Soundness (Spec/Equivalence.lean)

```
1 theorem operational_to_declarative
2   {G : Database} {consts : ConstSet} {fr : Frame} {e : Expr}
3   (h_wf : WellFormedDatabaseStrong G consts)
4   (h_fr_disjoint : Spec.FrameVarsDisjointConsts consts fr) :
5   Provable G fr e ->
6   Declarative.Provable (dbToAxioms G) (frameToContext fr)
7   (exprToFormula (varMapOfFrame fr) e)
```

Listing 7: Soundness: operational acceptance implies semantic validity.

Completeness at one frame (Spec/Equivalence.lean)

```
1 theorem frameDerivable_to_proofValid
2   {G : Database} {consts : ConstSet} {fr : Frame} {e : Expr}
3   (h_wf : WellFormedDatabaseStrong G consts)
4   (h_fr_nodup : FloatVarNoDup fr)
5   (h_fr_disjoint : Spec.FrameVarsDisjointConsts consts fr)
6   (h_mario : FrameDerivable G fr (exprToFormula (varMapOfFrame fr) e)) :
7   Provable G fr e
```

Listing 8: Completeness: frame derivability implies operational acceptance.

The FrameDerivable condition. Mario’s `Provable` relation includes a `var` constructor that requires no hypothesis: any variable is trivially derivable in an unbounded context. The operational verifier, however, works with a finite frame map and can only accept variables that are explicitly typed by floating hypotheses.

`FrameDerivable` is `Provable` where every `var` leaf must be a variable of the frame’s floating hypothesis map: derivability by Metamath’s rules in that frame, which is exactly what the verifier accepts. It is the instance of `Derivable` `axs T` (`Spec/Derivable.lean`), Mario’s `Provable` with a set `T` of typed variables, which is the closure of a *pre-statement* in the Metamath book’s Appendix C. The book defines a statement to be provable if it is the reduct of a provable pre-statement; `Statement.provable_iff_exists_extension` proves that this is Mario’s `Statement.Provable`. Mario’s `Provable` is strictly stronger at one fixed frame: `Spec/FixedFrameCounterexample.lean` gives well-formed databases in which a statement is provable in Mario’s semantics but has no proof in its own frame, because a dummy variable or an optional `$d` statement is missing. The Metamath book (§4.2.7) lets a proof use an *extended frame*: the frame plus optional `$f` and `$d` statements for dummy variables. `statementProvable_iff_exists_extendedFrame` proves that a stored statement is provable in Mario’s statement-level semantics (`Statement.Provable`, which uses `Statement.untrim`) iff some extended frame of it has a Metamath proof.

Premises. Both theorems share three structural conditions:

- `WellFormedDatabaseStrong`: every assertion in the database has well-formed hypotheses, consistent typecodes, and valid DV pairs.
- `FloatVarNoDup`: floating hypotheses in the frame have distinct variables (completeness only).
- `FrameVarsDisjointConsts`: frame variables do not collide with database constants.

All three are discharged automatically from parser-success hypotheses in `ParserEquivalence.lean`, so they do not appear as obligations at call sites.

Stored statements. `statementProvable_iff_exists_extendedFrame` (`Spec/Completeness.lean`) combines both directions for stored statements: soundness through frame trimming (`statementProvable_of_frameReduction`) and completeness by renaming the finitely many variables a derivation uses outside the frame into fresh dummy variables, one `$d` pair per pair of variables involving a dummy (`exists_dummies_of_statementProvable`).

The checker at a database state. At a parser state between statements, the stored statement of a `$p` claim is provable in Mario’s semantics iff, after declaring finitely many fresh dummy variables with the parser’s own `$v`, `$f` and `$d` actions, the parser accepts a normal-mode proof of the claim (frame trimming, the proof steps, and `finishProof`); this is `acceptedWithDummies_iff_statementProvable` in `CheckerCompleteness.lean`. The declarations here are database actions; `SourceCompleteness.lean` realizes them, and the proof, as source text (§3).

Bridge functions. The equivalence uses four translation functions:

- `dbToAxioms`: maps the operational `Database` (a partial function from labels to frame-expression pairs) to a predicate on `Statements`.
- `frameToContext`: converts an operational `Frame` to a declarative `Context` (hypothesis list + DV relation).
- `exprToFormula`: converts an operational `Expr` to a declarative `Formula` (typecode-term pair).
- `varMapOfFrame`: extracts the variable-to-typecode mapping from floating hypotheses in the frame.

A reviewer should check that these translations faithfully model the Metamath specification (§§4.1–4.2 of [1]).

5 Prefix Provenance

5.1 The Circularity Problem

When building a database incrementally (reading a `.mm` file top to bottom), each `$p` theorem is inserted *after* its proof is checked. But can we be sure a theorem doesn’t use itself—or a later theorem—in its own proof?

Under the default Metamath configuration, the answer is *no*: the `?` step mechanism (section 4.4.6 of [1]) allows incomplete proofs that push the theorem’s own formula onto the stack without any real proof. This is by design—`?` marks a proof as incomplete—but it means the parser cannot guarantee that accepted proofs are genuinely sound in the pre-insertion database.

5.2 The IsSound Predicate

We define a configuration predicate that is sufficient for prefix provenance:

```
1 def IsSound (c : ModeConfig) : Prop :=
2   c.rejectUnknownSteps = true
3   and c.allowDuplicateFloat = false
```

Listing 9: IsSound (Verify.lean).

Two concrete configurations satisfy this:

- `sound`: minimal strict mode (`rejectUnknownSteps := true`).
- `knife`: stricter mode (also rejects top-level essential hypotheses).

5.3 The Prefix Provenance Theorem

```
1 theorem checkBytes_finalState_finishProofEvent_certified
2   (arr : ByteArray) (config : ModeConfig)
3   (h_cfg : config.IsSound)
4   (h_success : (checkBytes arr config).error? = none) :
5   let s0 : ParserState := { (default : ParserState) with
6     db := { (default : DB) with config := config } }
7   forall (pos : Nat) (tk : ByteSlice) (pr : ProofState),
8     FinishProofEvent (s0.feedAll 0 arr) pos tk pr ->
9     exists (G : Spec.Database) (fr : Spec.Frame),
10      toDatabase (s0.feedAll 0 arr).db = some G
11      and toFrame (s0.feedAll 0 arr).db
12        (s0.feedAll 0 arr).db.frame = some fr
13      and Spec.Provable G fr (toExpr pr.fmla)
```

Listing 10: Prefix provenance (PrefixProvability/Checker.lean).

Reading the theorem. For every `FinishProofEvent` (a moment during parsing when a `$p` proof is completed), the theorem’s formula is `Spec.Provable` in the database and frame at the moment of the event—*before* the theorem is inserted. This rules out circular reasoning. `check_storedStatements_writtenProof` (`StoredStatementSoundness.lean`) supplies local insertion-event witnesses for stored assertions in an `IsSound` mode; its conclusion does not retain membership of those witnesses in the actual run. `check_every_theorem_provable_from_run_axiom_events` (`RunEmission.lean`) retains the actual-run membership and `$a/$p` classification, and eliminates earlier theorems by cut: every stored theorem is provable from the run’s `$a` statements.

Proof technique: ghost predicates. The proof maintains a `ProofGhost` invariant through the parser loop. This ghost tracks the proof state’s stack and heap certificates, the preload phase structure (for compressed proofs), and a scope condition (that the proof frame’s hypotheses are a subset of the database frame). The ghost is established when a `$p` token starts a proof and maintained through every `feedToken` call. At `finishProof`, the ghost is discharged to produce `Spec.Provable`.

6 What Is Not Claimed

Honest verification requires documenting not just what is proved but also what is *not* proved.

Include expansion. The default checker path is single-pass include handling (`check`) with bounded depth and explicit include errors. The parser/driver split keeps parser stepping pure and isolates IO in the include-dispatch loop. The bridge between `check` and `checkBytes` is proved for a single root file whose run raises no include request; completeness for source text with include directives is not claimed, and the filesystem calls are trusted.

Error codes: consistency only. The theorem `checkBytes_parseErrorCode?_fullyCertified` (Listing A.4, Appendix A) proves that each of the 65 error codes agrees with the evidence the parser recorded: the evidence has the code’s tag and payload shape, and the rule predicate of the code holds of it. These predicates are stated over the recorded evidence, so the theorem certifies the consistency of each diagnostic; it does not prove, independently of that evidence, that the source violates the rule. Nor do we prove that “if violation *X* is the first to occur, the parser produces code *X*.” First-violation completeness would require formalizing execution-order priority among checks—tractable but not essential for correctness review.

The parser’s reading is tested, not proved, book-conformant. All theorems concern source text as the parser of `Verify.lean` reads it. No theorem states that the parser accepts exactly the files the Metamath book accepts; the specification suite and the reference differential test it. In the `zar`, `sound` and `knife` modes the parser rejects a comment that holds a byte outside the book’s character set (book §4.1.1) or `$(` or `$)` inside a token (book §4.1.2); the `exe` mode treats vertical tab as whitespace, as `metamath.exe` does, and the `permissive` mode ignores every comment byte and inner `$(` until the first standalone `$)`. Comments do not nest, and an unterminated comment is rejected in every mode. All modes intentionally permit different `$f` types for a variable in separate scopes (`test59`), rather than enforce the global-type rule of book §4.1.3. Each stored assertion retains its own frame; an application still has to satisfy its typing premises. This is separate from the mode-dependent policy on simultaneously active duplicate `$f` declarations. The book’s character set does not make raw terminal display trustworthy: a carriage return, which the book allows, can overwrite a displayed line.

Normal proofs, without a size bound. The source-level theorems produce a normal proof, and no bound on its size is claimed: a normal proof can be exponentially longer than a compressed one. Compressed proofs are related to normal ones at the database level by `ProofReachableZ_iff_NormalProofReachable`.

Existence, not a proof generator. The completeness theorems are existence statements in `Prop`. They do not compute the dummy declarations or the proof.

Provability relative to the prefix. The source-level theorems relate a claim to the assertions read so far. In modes that accept `?` steps, these include earlier theorems whose proofs are incomplete. Provability from the `$a` statements alone is proved for complete runs in `IsSound` modes (§5), not for a source prefix.

Mode boundary. The source-level theorems assume a mode without duplicate `$f` statements, as §4.1.3 of the book requires. The `exe` and `permissive` presets allow such statements and are outside these theorems.

Mode-indexed vacuous predicates. Some predicates (`TokpInv`, `ProofGhost`, etc.) use `| _ => True` for modes where the invariant is intentionally vacuous (e.g. `TokpInv` is only constrained in proof mode). These are *not* placeholder theorems or masked `sorry`s—they are deliberate “this mode imposes no constraint” defaults.

toFrame returns Option. The translation `toFrame : DB -> Frame -> Option Spec.Frame` is genuinely partial: not every implementation `Frame` corresponds to a well-formed spec `Frame`. The main biconditional existentially quantifies over a successful `toFrame` extraction.

No performance claims. Heartbeat budgets and `set_option maxHeartbeats` annotations are engineering accommodations for Lean’s kernel, not mathematical claims.

7 Reproducing Mechanically

7.1 Build Instructions

- **Lean toolchain:** `leanprover/lean4:v4.33.1` (specified in `lean-toolchain`)
- **Dependencies:** Batteries at commit `4488d40` (no Mathlib)

```
# Build the full target set
lake build Metamath MetamathExperimental MetamathLegacy mm-lean4 \
  validateDB testCliErrorCode testCheckSinglePassParity testParserInvariants
# -> Build completed successfully.

# Run parity canary (single-pass vs legacy boundary)
./lake/build/bin/testCheckSinglePassParity
# -> parity checks pass

# Run the parser-invariant tests
./lake/build/bin/testParserInvariants
# -> all tests pass

# Run the metamath-test suite for mm-lean4
ROOT="$(pwd)"
cd "$METAMATH_TEST"
MM_LEAN4="$ROOT/lake/build/bin/mm-lean4" ./run-testsuite-all ./test-mm-lean4
# -> Score: 187/187 (100%)
```

7.2 Verifying Sorry-Freedom

```
rg -n '^s*(sorry|admit)\b|by (sorry|admit)\b' Metamath/ --glob '*.lean'
# -> no output
```

```

rg -n '^\\s*axiom\\s' Metamath/ --glob '*.lean'
# -> one line, inside a documentation comment of ParserInvariants.lean

lake env lean scripts/print_axioms.lean
# -> the listed theorems depend only on propext, Classical.choice
#    and Quot.sound

```

7.3 Reviewing Strategy

Start at `Metamath/SourceCompleteness.lean`, the entry module for the source-level theorems, and at `Metamath/ParserEquivalence.lean` for the proof-run and frame-derivability theorems; `docs/SpecTheoremMap.md` maps each clause of the specification to its theorems. From there:

1. Read the theorem *statements* (not proofs) of the main biconditionals.
2. Check the specifications: the operational one in `Spec/Operational.lean`, the declarative one in `DeclarativeSpec.lean`, and, in `Spec/DeclarativeOriginal.lean`, Mario Carneiro's original rule.
3. Check the bridge functions (`toDatabase`, `toFrame`, `toExpr`, `dbToAxioms`, `frameToContext`) and the rendered source text with its conditions (`render`, `Admissible`, `SourceAccepts`).
4. Run the full-target build command above to let Lean's kernel verify the maintained proof targets.

8 Architecture and Proof Strategy

8.1 Layer Diagram

The verification proceeds bottom-up through the following layers:

```

ByteArray
|
v
checkBytes (Verify.lean)
|
v
WellFormedDB (WellFormedness.lean)
|
v
verify_impl_sound / verify_impl_complete (KernelCorrectness.lean)
|
v
operational_to_declarative / frameDerivable_to_proofValid
  (Spec/Equivalence.lean)
|
v
proofChecker_normal_acceptance_iff_specProvable_in_parsedDB
  (KernelCorrectness.lean)
|

```

```

      v
proofChecker_normal_iff_frameDerivable_in_parsedDB
  (ParserEquivalence.lean)
  |
    v
acceptedWithDummies_iff_statementProvable (CheckerCompleteness.lean)
  |
    v
statementProvable_iff_sourceAccepts / _fileAccepts
  (SourceCompleteness.lean)

```

8.2 Theorem Chain

The proof proceeds through the following chain (all sorry-free):

1. `checkBytes` parses bytes and validates `wellFormed?` as a final gate.
2. `checkBytes_no_error_wellFormed?`: no error implies `wellFormed? = true`.
3. `wellFormedDB_of_wellFormed?`: `wellFormed? = true` implies `WellFormedDB`.
4. `parser_construction_wellformed`: composes steps 2 and 3.
5. `verify_impl_sound`: the soundness direction (a successful proof run implies provability).
6. `verify_impl_complete`: the completeness direction (provability implies a proof run whose final formula has the claim’s expression; `verify_impl_complete_exact` gives the literal final stack that `finishProof` requires).
7. `operational_to_declarative`: unconditional spec-level soundness.
8. `frameDerivable_to_proofValid`: completeness from frame derivability.
9. `proofChecker_normal_iff_frameDerivable_in_parsedDB`: parser-level frame-derivability biconditional.
10. The end-to-end biconditionals: combined from steps 5–8.
11. `statementProvable_iff_exists_extendedFrame`: completeness for Mario’s semantics, up to extended frames.
12. `acceptedWithDummies_iff_statementProvable`: the same against the parser’s acceptance at a database state between statements.
13. `statementProvable_iff_sourceAccepts`: the same for the source text the parser reads; `statementProvable_iff_fileAccepts` for the complete file under `checkBytes`.

8.3 Key Proof Techniques

Parser loop induction. The main soundness proof uses induction over the token stream, maintaining a `ParserStateInv` invariant that tracks well-formedness, frame scoping, and hypothesis consistency.

Ghost predicates. For prefix provenance, a `ProofGhost` predicate is maintained through the parser loop. The ghost carries four conjuncts tracking stack/heap certificates, the preload phase structure, and the scope condition for compressed proofs.

8.4 Statistics

Table 2: Project statistics.

Metric	Value
Lean lines under <code>Metamath/</code>	81,574
Lean files under <code>Metamath/</code>	103
Specification-suite databases	187 (all pass)
Sorries	0
Declared axioms	0
Error codes consistent with their evidence	65/65
Largest file	<code>ParserInvariantPreservation.lean</code> (11,750 lines)

9 Conclusion

We have presented a formal verification of a Metamath proof checker against Mario Carneiro’s declarative semantics. The central result—completeness and soundness for source text, also for complete files under `checkBytes` and root files under `check`—establishes that the implementation is sound and complete for source text as its parser reads it, with normal proofs as witnesses, in modes without duplicate `$f` statements. Proof runs in a parsed database are related to specification-level provability, and under strict configuration, prefix provenance rules out circular reasoning.

Related work. `metamath.exe`, `mmj2`, `metamath-knife` and `mmverify.py` are checked against shared test suites rather than proved correct; the differential harness of this project compares the verdicts of its `exe` and `knife` modes with those of `metamath.exe` and `metamath-knife`. Metamath Zero [2] is a related verification system designed with the goal of a verifier whose own correctness is proved. The Metamath specification [1] defines the proof system but does not itself come with a machine-checked proof of checker correctness.

Future directions. Three natural extensions remain:

- Error code *completeness*: proving that if a specific violation occurs first, the parser produces the corresponding code.
- A filesystem-oracle model for IO dispatch (`realPath`, `readBinFile`), and completeness for source text with include directives.
- Integration with mm0: bridging our Lean formalization to the mm0 proof format used by `mm0-rs`.

Acknowledgments

We thank Mario Carneiro for the Lean 4 Metamath verifier and declarative specification from which this development is derived.

A Key Theorem Listings

This appendix reproduces the Lean signatures of the main definitions and theorems, with Unicode notation written in ASCII.

A.1 Spec.ProofValid (Spec/Operational.lean)

```
1 inductive ProofValid (G : Database) :
2   Frame -> List Expr -> List ProofStep -> Prop where
3 | nil : forall fr, ProofValid G fr [] []
4 | useEssential : forall fr stack steps e,
5   Hyp.essential e in fr.hyps ->
6   ProofValid G fr stack steps ->
7   ProofValid G fr (e :: stack)
8   (ProofStep.useHyp (Hyp.essential e) :: steps)
9 | useFloating : forall fr stack steps c v,
10  Hyp.floating c v in fr.hyps ->
11  ProofValid G fr stack steps ->
12  ProofValid G fr (<c, [v.v]> :: stack)
13  (ProofStep.useHyp (Hyp.floating c v) :: steps)
14 | useAxiom : forall fr stack steps l fr' e s,
15  G l = some (fr', e) ->
16  dvOK fr.vars fr'.dv fr.dv s ->
17  (forall c v, Hyp.floating c v in fr'.hyps ->
18   (s v).typecode = c) ->
19  ProofValid G fr stack steps ->
20  forall needed : List Expr,
21  needed = fr'.hyps.map (fun h => match h with
22   | Hyp.essential e => applySubst fr'.vars s e
23   | Hyp.floating _ v => s v) ->
24  forall remaining : List Expr,
25  stack = needed.reverse ++ remaining ->
26  ProofValid G fr (applySubst fr'.vars s e :: remaining)
27  (ProofStep.useAssertion l s :: steps)
```

A.2 statementProvable_iff_exists_extendedFrame (Spec/Completeness.lean)

```
1 theorem statementProvable_iff_exists_extendedFrame
2   {G : Database} {consts : ConstSet} {fr : Frame} {e : Expr}
3   (hdb : WellFormedDatabaseStrong G consts)
4   (hfin : exists L : List String, forall s, consts s -> s in L)
5   (hfr : FrameWellFormed fr)
6   (hdc : FrameVarsDisjointConsts consts fr)
7   (hscope : FrameExprsInScope consts fr e) :
8   (statementOfFrame fr e).Provable (dbToAxioms G)
```

```
9    <-> exists fr', ExtendedFrame consts fr fr' and Spec.Provable G fr' e
```

A.3 proofChecker_normal_iff_frameDerivable_in_parsedDB (ParserEquivalence.lean)

```
1 theorem proofChecker_normal_iff_frameDerivable_in_parsedDB
2   (bytes : ByteArray)
3   (label : String)
4   (f : Verify.Formula)
5   (h_success : (Verify.checkBytes bytes).error? = none)
6   :
7   (... a normal proof run for label/f ...)
8   <->
9   (exists (fr : Spec.Frame),
10    toFrame (Verify.checkBytes bytes)
11    (Verify.checkBytes bytes).frame = some fr
12    and FrameDerivable
13    (toDatabaseTotal (Verify.checkBytes bytes))
14    fr
15    (exprToFormula (varMapOfFrame fr) (toExpr f)))
```

A.4 checkBytes_parseErrorCode?_fullyCertified (ErrorCodeSemantics.lean)

```
1 theorem checkBytes_parseErrorCode?_fullyCertified
2   (arr : ByteArray) (config : ModeConfig) (code : ParseErrorCode) :
3   (checkBytes arr config).parseErrorCode? = some code ->
4   HasCodeEvidence (checkBytes arr config) code
5   and (checkBytes arr config).RuleSemanticViolation code
```

A.5 ModeConfig.IsSound and sound (Verify.lean, Verify/Conformance.lean)

```
1 def sound : ModeConfig := {
2   rejectUnknownSteps := true
3 }
4
5 def IsSound (c : ModeConfig) : Prop :=
6   c.rejectUnknownSteps = true
7   and c.allowDuplicateFloat = false
8
9 theorem isSound_sound :
10   IsSound sound := <rfl, rfl>
```

B File Inventory

The authoritative inventory is all .lean files under Metamath/. At the time of writing:

- Lean files under Metamath/: 103

- Total lines under Metamath/: 81,574
- Largest file: Metamath/ParserInvariantPreservation.lean (11,750 lines)

For a full reproducible inventory snapshot:

```
find Metamath -name "*.lean" | sort
find Metamath -name "*.lean" -print0 | xargs -0 wc -l
```

C Unit Tests and Spec Alignment

This appendix lists unit tests from the `metamath-test` test suite, grouped by the Metamath specification rule they exercise. Each test shows the expected verdict (ACCEPT or REJECT) and the complete `.mm` file content. Tests exceeding 70 lines are summarized in a table. Descriptions are quoted from the manifest, including its own section labels; subsection titles give the sections of the Metamath book. The test manifest (`run-testsuite-all`) is the authoritative source; this appendix is auto-generated from it.

C.1 §4.1.1 Preliminaries

demo0-illegal-label-bad1.mm [REJECT] *Sec. 4.1.1: labels must be alphanumeric + hyphen + underscore + period*

```
$( demo0.mm 1-Jan-04 $)

$(
    PUBLIC DOMAIN DEDICATION

    This file is placed in the public domain per the Creative Commons Public
    Domain Dedication. ht\tp://creativecommons.org/licenses/publicdomain/

    Norman Megill - email: nm at alum.mit.edu

$)

$( This file is the introductory formal system example described
    in Chapter 2 of the Metamath book. $)

$( Declare the constant symbols we will use $)
    $c 0 + = -> ( ) term wff |- $.
$( Declare the metavariables we will use $)
    $v t r s P Q $.
$( Specify properties of the metavariables $)
    t\t $f term t $.
    tr $f term r $.
    ts $f term s $.
    wp $f wff P $.
    wq $f wff Q $.
$( Define "term" (part 1) $)
    (tze $a term 0 $.
$( Define "term" (part 2) $)
    t)pl $a term ( t + r ) $.
$( Define "wff" (part 1) $)
    w"eq $a wff t = r $.
$( Define "wff" (part 2) $)
    w%i\m $a wff ( P -> Q ) $.
$( State axiom a1 $)
    a1 $a |- ( t = r -> ( t = s -> r = s ) ) $.
$( State axiom 'a2' $)
    'a2' $a |- ( t + 0 ) = t $.
```

```

${
  min $e |- P $.
  maj $e |- ( P -> Q ) $.
$( Define the modus ponens inference rule $)
  mp $a |- Q $.
$}
$( Prove a theorem $)
th1 $p |- t = t $=
$( Here is its proof: $)
  t\t (tze t)pl t\t w"eq t\t t\t w"eq t\t 'a2' t\t (tze t)pl
  t\t w"eq t\t (tze t)pl t\t w"eq t\t t\t w"eq w%i\m t\t 'a2'
  t\t (tze t)pl t\t t\t a1 mp mp
$.

```

emptyline.mm [ACCEPT] *Sec. 4.1.1: whitespace (including newlines) is valid separator*

comment1.mm [REJECT] *Sec. 4.1.1: whitespace required between tokens*

```

$( $( $)

```

underscores.mm [ACCEPT] *Sec. 4.1.1: underscores allowed in labels*

```

$c a $.

$( This is _italic_ and the command "MM-PA> MINIMIZE__WITH *" and a
sub_script, a label ~ lab[["'~~__ , and a url: ~ http://a_b__c.com/[["'~~__
Doubled chars: [[" ' ' ~~ __
<HTML>
Inside HTML tags:

This is _italic_ and the command "MM-PA> MINIMIZE__WITH *" and a
sub_script, a label ~ lab[["'~~__ , and a url: ~ http://a_b__c.com/[["'~~__
Doubled chars: [[" ' ' ~~ __
</HTML> $)
underscores $a a $.

$( $t
htmldef "a" as "a";
alhtmldef "a" as "a";
latexdef "a" as "a"; $)

```

test01_non-printable_ascii_characters.mm [REJECT] *Sec. 4.1.1: only printable ASCII + whitespace allowed*

```

$( Unit Test 1: Non-printable ASCII characters $)
$( Should reject: True $)

$c wff |- $.
$c <0xE2><0x86><0x92> $.
$v x $.
wf $f wff x $.
ax $a |- x $.

```

test02_missing_whitespace_between_tokens.mm [REJECT] *Sec. 4.1.1: tokens must be separated by whitespace*

```

$( Unit Test 2: Missing whitespace between tokens $)
$( Should reject: True $)

$cwff$c|-.
$vx$.
wf$fwffx$.
ax$a|-x$.

```

test06_dangling_dollar_sign_at_eof.mm [REJECT] *Sec. 4.1.1: \$ must be followed by a valid keyword character*

```
$( Unit Test 6: Dangling dollar sign at EOF $)
$( Should reject: True $)

$c wff |- $.
$v x $.
wf $f wff x $.
$
```

test18_missing_whitespace_after_comment.mm [REJECT] *Sec. 4.1.1: whitespace required between tokens*

```
$( Unit Test 18: Missing whitespace after comment $)
$( Should reject: True $)

$c wff $.
$v x $.
$( No space after comment close $)wf $f wff x $.
```

test25_missing_space_before_comment.mm [REJECT] *Sec. 4.1.1, 4.1.2: \\$(must be whitespace-separated*

```
$( Unit Test 25: Missing space before comment $)
$( Should reject: True $)

$c wff$. $(comment$)
```

test31_missing_whitespace_after_comment.mm [REJECT] *Sec. 4.1.1, 4.1.2: \\$(must be whitespace-separated*

```
$( Unit Test 31: Missing whitespace after comment $)
$( Should reject: True $)

$( comment $)$c wff $.
```

test49_token_splice_axiom.mm [REJECT] *Sec. 4.1.1, 4.1.2: token boundaries must be preserved*

```
$( Unit Test 49: Include as token splice in axiom $)
$( Should reject: True - include inside statement is invalid in strict mode $)
$( Category: CORE - Strict include semantics $)
$( Spec Section 4.1.2: "include directives are processed at outermost level" $)
$( Spec Appendix E: Include not allowed inside statements $)

$c wff |- $.
$v x $.
fx $f wff x $.

$( Include splices tokens into the middle of this statement - INVALID $)
ax-splice $a $[ ./helpers/test49_helper.mm $]
```

test50_token_splice_proof.mm [REJECT] *Sec. 4.1.1, 4.1.2: token boundaries must be preserved*

```
$( Unit Test 50: Include as token splice in proof $)
$( Should reject: True - include inside statement is invalid in strict mode $)
$( Category: CORE - Strict include semantics $)
$( Spec Section 4.1.2: "include directives are processed at outermost level" $)
$( Spec Appendix E: Include not allowed inside statements $)

$c wff |- $.
$v x $.
fx $f wff x $.
ax-x $a |- x $.

$( Include splices proof steps from external file - INVALID $)
th $p |- x $= $[ ./helpers/test50_helper.mm $] $.
```

test54_whitespace_variety.mm [ACCEPT] *Sec. 4.1.1: various whitespace characters are valid*

```
$( Test 54: Whitespace variety - tabs and spaces are valid whitespace per Sec 4.1.1; should ACCEPT $)
```

```

$c wff |- $.
$v ph $.

f1 $f wff ph $.
a1 $a |- ph $.
p1 $p |- ph $= f1 a1 $.

```

test68_formfeed_whitespace.mm [ACCEPT] *Sec. 4.1.1: form-feed (0x0C) is whitespace*

```

$( Test 68: Form-feed (0x0C) is valid whitespace per Metamath spec section 4.1.1; should ACCEPT $)

$c<0x0C>wff<0x0C>|-<0x0C>$ .
$v<0x0C>ph<0x0C>$ .

f1<0x0C>$f<0x0C>wff<0x0C>ph<0x0C>$ .
a1<0x0C>$a<0x0C>|-<0x0C>ph<0x0C>$ .
p1<0x0C>$p<0x0C>|-<0x0C>ph<0x0C>$=<0x0C>f1<0x0C>a1<0x0C>$ .

```

test110_comment_control_byte.mm [REJECT] *Sec. 4.1.1, 4.1.2: a comment may contain only printable ASCII and whitespace (BEL)*

```

$( Unit Test 110: BEL (byte 7) inside a comment $)
$( Should reject: True $)

$c wff |- $.
$v x $.
wf $f wff x $.
ax $a |- x $.
$( bell<0x07>here $)

```

test111_comment_vertical_tab.mm [REJECT] *Sec. 4.1.1, 4.1.2: vertical tab is not Metamath whitespace (metamath.exe accepts)*

```

$( Unit Test 111: vertical tab (byte 11) inside a comment $)
$( Should reject: True $)

$c wff |- $.
$v x $.
wf $f wff x $.
ax $a |- x $.
$( tab<0x0B>here $)

```

test112_comment_non_ascii.mm [REJECT] *Sec. 4.1.1, 4.1.2: a comment may not contain non-ASCII bytes (metamath-knife 0.3.9 panics)*

```

$( Unit Test 112: UTF-8 bytes inside a comment $)
$( Should reject: True $)

$c wff |- $.
$v x $.
wf $f wff x $.
ax $a |- x $.
$( caf<0xC3><0xA9> $)

```

test115_vertical_tab_between_tokens.mm [REJECT] *Sec. 4.1.1: vertical tab does not separate tokens (metamath.exe accepts)*

```

$( Unit Test 115: vertical tab between tokens $)
$( Should reject: True $)

$c wff |-<0x0B>$ .
$v x $.
wf $f wff x $.
ax $a |- x $.

```

test116_comment_escape_hides_axiom.mm [REJECT] *Sec. 4.1.1, 4.1.2: terminal escape bytes*

in a comment can hide an axiom on a terminal

```
$( Unit Test 116: terminal escape bytes in a comment after an axiom $)
$( Should reject: True $)

$c wff |- F $.
$v x $.
wf $f wff x $.
evil $a |- F $. $( <0x1B>[2K<0x1B>[G$( see the axioms above $)
th $p |- F $= evil $.
```

test57_case_sensitivity_labels.mm [ACCEPT] *Sec. 4.1.1, 4.2.1: labels are case-sensitive*

```
$( Test 57: Case sensitivity for labels - labels are case-sensitive per Sec 4.1.3; should ACCEPT $)

$c wff |- $.
$v ph $.

f1 $f wff ph $.
ax $a |- ph $.
Ax $a |- ph $.

p1 $p |- ph $= f1 ax $.
p2 $p |- ph $= f1 Ax $.
```

C.2 §4.1.2 Preprocessing

test03_nested_comment_delimiters.mm [REJECT] *Sec. 4.1.2: \\$(may not appear inside comments*

```
$( Unit Test 3: Nested comment delimiters $)
$( Should reject: True $)

$c wff |- $.
$( outer $( nested $) comment $)
$v x $.
wf $f wff x $.
ax $a |- x $.
```

test17_include_scope_violation.mm [REJECT] *Sec. 4.1.2: \\$/\\$\$ scope rules*

```
$( Unit Test 17: Include inside block with scope violation $)
$( Should reject: True $)

$c wff |- $.
$v x $.
wx $f wff x $.

${
  $( Include inside block - contents scoped to block $)
  $[ ./helpers/test17_helper.mm $]

  $( This would work - using inside the block $)
  $( th1 $p |- y $= wy ax-inner $. $)
$}

$( Try to use ax-inner outside the block - SCOPE VIOLATION $)
th2 $p |- y $= wy ax-inner $.
```

test28_self_include.mm [ACCEPT] *Sec. 4.1.2: a self-reference is ignored (spec: to avoid loops)*

```
$( Unit Test 28: Self-include $)
$( Should reject: False - Spec section 4.1.2 says "will simply be ignored" $)
$( Note: metamath.exe REJECTS (spec divergence), mmverify_pure.py ACCEPTS (spec-compliant) $)

$c wff $.
```

```
$( Include this file itself - causes duplicate declarations $)
$[ ./test28_self_include.mm $]
```

test40_include_scope_correct_outer.mm [REJECT] *Sec. 4.1.2: `\$[\$]` only allowed in outermost scope*

```
$( Unit Test 40: Include inside block $)
$( Should reject: True - include must be at outermost level $)
$( Category: CORE - Strict include semantics $)
$( Spec Section 4.1.2: Include directives processed at outermost level only $)

$c wff |- $.
$v x $.
$w $f wff x $.
$ax-x $a |- x $.

${
  $( Include inside block $)
  $[ ./helpers/test40_helper.mm $]

  $( Use included axiom INSIDE the block - this should work $)
  $th1 $p |- y $= wy ax-inner $.
$}

$( Outside the block, only x is available $)
$th2 $p |- x $= wx ax-x $.
```

test42_include_duplicate_main.mm [ACCEPT] *Sec. 4.1.2: duplicate includes are ignored (like whitespace)*

```
$( Unit Test 42: Duplicate include - same file twice $)
$( Should reject: False - second include should be ignored $)
$( Spec: First inclusion is processed, subsequent inclusions ignored $)

$c wff |- $.

$( First include - processes the file $)
$[ ./helpers/test42_helper.mm $]

$( Second include - should be ignored (file already seen) $)
$[ ./helpers/test42_helper.mm $]

$( Use declarations from first include $)
$th1 $p |- var-dup $= fdup ax-dup $.
```

test44_include_cycle_main.mm [ACCEPT] *Sec. 4.1.2: a cyclic include is a later reference, ignored*

```
$( Unit Test 44: Cycle detection - A includes B, B includes A $)
$( Should reject: False - cycle should be detected and ignored $)
$( Spec: Include stack tracking prevents infinite loops $)

$c wff |- $.

$( Include A, which includes B, which tries to include A again $)
$[ ./helpers/test44_helper_a.mm $]

$( Both A and B should be included exactly once $)
$( Use declarations from both files $)
$th-a $p |- var-a $= fa-cycle ax-a-cycle $.
$th-b $p |- var-b $= fb-cycle ax-b-cycle $.
```

test46_duplicate_include_main.mm [REJECT] *Sec. 4.1.2: include directives must be outermost*

```
$( Unit Test 46: Duplicate include in different inner blocks $)
$( Should reject: True - includes must be at outermost level $)
$( Category: CORE - Strict include semantics $)
$( Spec Section 4.1.2: Include directives processed at outermost level only $)
```

```

$c wff |- $.

${
  $( First include: processes the file $)
  $[ ./helpers/test46_helper.mm $]

  $( Use the included content $)
  th1 $p |- y $= wy ax-y $.
$}

${
  $( Second include: should be ignored as whitespace $)
  $[ ./helpers/test46_helper.mm $]

  $( This would fail if file processed twice (y redeclared) $)
  $( But since second include ignored, ax-y and wy are still available globally $)
  th2 $p |- y $= wy ax-y $.
$}

```

test52_include_inside_e.mm [REJECT] *Sec. 4.1.2: \\${ not allowed inside a statement*

```

$( Test 52: $[ ... $] inside an $e statement - must REJECT per Sec 4.1.2 (includes only at outermost level) $
↪ )

$c wff |- $.
$v ph $.

e1 $e $[ ./helpers/test40_include_scope_correct_inner.mm $] $.

```

test53_include_inside_d.mm [REJECT] *Sec. 4.1.2: \\${ not allowed inside a statement*

```

$( Test 53: $[ ... $] inside a $d statement - must REJECT per Sec 4.1.2 (includes only at outermost level)
↪ and Sec 4.2.4 (varlist only) $)

$c wff |- $.
$v x y $.

$d $[ ./helpers/test40_include_scope_correct_inner.mm $] x y $.

```

test74_dollar_d_comment_interleaved.mm [ACCEPT] *Sec. 4.1.2: a comment inside \\${ does not break accumulation*

```

$( Unit Test 74: comment inside a $d statement $)
$( Should reject: False $)
$( Sec. 4.1.2: comments are whitespace and may appear between any two
tokens; the parser must resume the $d accumulation afterwards, so the
statement still reaches the terminator with two variables. $)

$c wff $.
$v x y $.
$d x $( interleaved comment $) y $.

```

test87_include_child_unclosed_var.mm [REJECT] *Sec. 4.1.2: an included file may not end mid-statement*

```

$( Unit Test 87: an included file may not end mid-statement (Sec. 4.1.2) $)
$c wff $.
$[ ./helpers/test87_helper.mm $]
x $.

```

test88_include_child_unclosed_comment.mm [REJECT] *Sec. 4.1.2: an included file may not end inside a comment*

```

$( Unit Test 88 $)
$c wff $.
$[ ./helpers/test88_helper.mm $]
$)

```

test89_include_child_unclosed_block.mm [REJECT] *Sec. 4.1.2: an included file may not end with a block open*

```
$( Unit Test 89 $)
$c wff $.
$[ ./helpers/test89_helper.mm $]
$}
```

test90_include_child_unclosed_axiom.mm [REJECT] *Sec. 4.1.2: an included file may not end mid-\\$a*

```
$( Unit Test 90 $)
$[ ./helpers/test90_helper.mm $]
$.
```

test91_include_child_unclosed_proof.mm [REJECT] *Sec. 4.1.2: an included file may not end mid-\\$p*

```
$( Unit Test 91 $)
$[ ./helpers/test91_helper.mm $]
$.
```

test92_include_child_balanced_block.mm [ACCEPT] *Sec. 4.1.2: a child may open and close its own block*

```
$( Unit Test 92: a child may open and close its own block $)
$c wff $.
$[ ./helpers/test92_helper.mm $]
```

test93_include_child_unclosed_include.mm [REJECT] *Sec. 4.1.2: an included file may not end inside an unfinished \\$/*

```
$( Unit Test 93: a child may not end inside an unfinished \$/ $)
$c wff $.
$[ ./helpers/test93_helper.mm $]
$]
```

test94_include_child_empty.mm [ACCEPT] *Sec. 4.1.2: an empty included file is a legal boundary*

```
$( Unit Test 94: an empty included file is a legal boundary $)
$c wff $.
$[ ./helpers/test94_helper.mm $]
```

test95_include_child_comment_only.mm [ACCEPT] *Sec. 4.1.2: a child holding only a complete comment is a legal boundary*

```
$( Unit Test 95: a child holding only a complete comment is a legal boundary $)
$c wff $.
$[ ./helpers/test95_helper.mm $]
```

test96_include_nested_origins.mm [ACCEPT] *Sec. 4.1.2: nested includes; \\$a and \\$p may close exactly at file boundaries*

```
$( Unit Test 96: nested includes with $a and $p origins at file boundaries $)
$c |- wff ph $.
$v P $.
wph $f wff P $.
$[ ./test96_child.mm $]
$( Duplicate include is suppressed, not re-processed $)
$[ ./test96_child.mm $]
th2 $p |- P $= wph ax96 $.
```

test113_comment_embedded_open.mm [REJECT] *Sec. 4.1.2: a comment may not contain \\$(/ inside a token*

```
$( Unit Test 113: dollar and open parenthesis inside a comment token $)
$( Should reject: True $)
```

```

$c wff |- $.
$v x $.
wf $f wff x $.
ax $a |- x $.
$( x$(y $)

```

test114_comment_embedded_close.mm [REJECT] *Sec. 4.1.2: a comment may not contain $\backslash \$$ inside a token*

```

$( Unit Test 114: dollar and close parenthesis inside a comment token $)
$( Should reject: True $)

$c wff |- $.
$v x $.
wf $f wff x $.
ax $a |- x $.
$( x$(y $)

```

C.3 §4.2.1 User-Defined Tokens

demo0-dollar-in-math-symbol-bad1.mm [REJECT] *Sec. 4.2.1: math symbols must not contain $\$$*

```

$( demo0.mm 1-Jan-04 $)

$(
    PUBLIC DOMAIN DEDICATION

    This file is placed in the public domain per the Creative Commons Public
    Domain Dedication. http://creativecommons.org/licenses/publicdomain/

    Norman Megill - email: nm at alum.mit.edu

$)

$( This file is the introductory formal system example described
    in Chapter 2 of the Meamath book. $)

$( Declare the constant symbols we will use $)
    $c 0 + = -> ( ) meep$ term wff |- $.
$( Declare the metavariables we will use $)
    $v t r s P Q $.

```

test05_dollar_sign_in_math_symbols.mm [REJECT] *Sec. 4.2.1: math symbols must not contain $\$$*

```

$( Unit Test 5: Dollar sign in math symbols $)
$( Should reject: True $)

$c wff |- $.
$c a$b $.
$v x $.
wf $f wff x $.
ax $a |- x $.

```

test34_label_token_in_math_context.mm [REJECT] *Sec. 4.2.1, 4.2.5: labels and math symbols are disjoint*

```

$( Unit Test 34: Label token in math context $)
$( Should reject: True $)

$c wff |- $.
$v x $.

```

```

wf $f wff x $.
$( Using 'wf' where math symbol expected $)
bad $a wf x $.

```

test58_case_sensitivity_symbols.mm [ACCEPT] *Sec. 4.2.1: math symbols are case-sensitive*

```

$( Test 58: Case sensitivity for symbols - constants are case-sensitive per Sec 4.1.3; should ACCEPT $)

$c WFF PH |- $.
$v ph $.

f1 $f WFF ph $.
a1 $a |- ph $.
p1 $p |- ph $= f1 a1 $.

```

C.4 §4.2.2 Constants and Variables

test80_variable_redeclared_while_active.mm [REJECT] *Sec. 4.2.2: a variable may not be redeclared while active*

```

$v x $.
$v x $.

```

test82_inactive_variable_in_float.mm [REJECT] *Sec. 4.2.2: a math symbol must be active, not merely declared*

```

$c wff $.
${
$v x $.
$}
fx $f wff x $.

```

test83_variable_reactivated_after_block.mm [ACCEPT] *Sec. 4.2.2: a variable may be redeclared once its block has been popped*

```

${
$v x $.
$}
$v x $.

```

test84_outer_var_survives_inner_pop.mm [ACCEPT] *Sec. 4.2.2: a top-level variable stays active after an inner block closes*

```

$( A top-level variable stays active after an inner block closes. $)
$c wff $.
$v x $.
${
$v y $.
$}
fx $f wff x $.

```

test85_nested_var_survives_deeper_pop.mm [ACCEPT] *Sec. 4.2.2: an outer-block variable stays active after a deeper block closes*

```

$( A variable declared in an outer block stays active after a deeper
  nested block closes. $)
$c wff $.
${
$v x $.
${
$v y $.
$}
fx $f wff x $.
$}

```

test86_sibling_block_reactivation.mm [ACCEPT] *Sec. 4.2.2: the same name may be declared in sibling blocks*

```
$( The same variable name may be declared in sibling blocks, since each
  declaration becomes inactive when its own block closes. $)
$c wff $.
${
  $v x $.
  fx1 $f wff x $.
}$
${
  $v x $.
  fx2 $f wff x $.
}$
```

C.5 §4.2.3 The \$c and \$v Declaration Statements

local-var-bad.mm [REJECT] *Sec. 4.2.3, 4.2.8: variable scope violation*

```
$c formula $.
$c A. $.
$c setvar $.
$v ph $.
wph $f formula ph $.

${
  $v x $.
  $( This is not valid because there is no $f for the variable 'x' $)
  $( Extend formula definition to include the universal quantifier ('for all'). $)
  wal $a formula A. x ph $.
}$
```

test07_redeclaration_of_constant.mm [REJECT] *Sec. 4.2.3, 4.2.8: constant may not be redeclared*

```
$( Unit Test 7: Redclaration of constant $)
$( Should reject: True $)

$c wff |- $.
$c wff $.
$v x $.
wf $f wff x $.
ax $a |- x $.
```

test45_variable_redeclaration.mm [ACCEPT] *Sec. 4.2.3, 4.2.8: variables may be redeclared in a new scope*

```
$( Unit Test 45: Variable redeclaration across scopes $)
$( Should reject: False - variable can be redeclared after scope ends $)
$( Spec Section 4.2.8: "A variable may be declared again after it becomes inactive" $)

$c wff class |- $.
$v x $.

${
  $( First scope: x has typecode wff $)
  fx1 $f wff x $.
  ax1 $a |- x $.
  $( Use x within its scope $)
  th1 $p |- x $= fx1 ax1 $.
}$

${
  $( Second scope: x has typecode class - VALID redeclaration $)
  fx2 $f class x $.
  ax2 $a class x $.
```

```

$( Use x within its scope $)
th2 $p class x $= fx2 ax2 $.
$}

```

test48_variable_conflict_main.mm [REJECT] *Sec. 4.2.3, 4.2.8: variable conflicts with constant*

```

$( Unit Test 48: Include causing variable redeclaration conflict $)
$( Category: CORE - CRITICAL scoping test $)
$( Should reject: True - x already active when include tries to redeclare it $)
$( Spec Section 4.2.8: "A variable may not be declared a second time while it is active" $)

$c wff |- $.
$v x $.
$fx $f wff x $.

$( Include file that also declares $v x - should fail $)
$[ ./helpers/test48_helper.mm $]

```

C.6 §4.2.4 The \$d Statement

test09_d_with_non-variables.mm [REJECT] *Sec. 4.2.4: \ \$d must contain only variables*

```

$( Unit Test 9: $d with non-variables $)
$( Should reject: True $)

$c wff |- $.
$d wff |- $.

```

test69_djvars_error_masking.mm [REJECT] *Sec. 4.2.4: \ \$d symbols must be declared variables*

```

$( Test 69: $d with undeclared symbol should reject. $)
$( This currently exposes a parser diagnostic masking path in mm-lean4:
the immediate token error can be overwritten by EOF unclosed-$d reporting. $)

$c wff $.
$v x $.

$d x y $.

```

test70_djvars_before_float.mm [ACCEPT] *Sec. 4.2.4: \ \$d requires declared variables, not prior \ \$f declarations*

```

$( Test 70: $d may appear before corresponding $f hypotheses. $)

$c wff |- $.
$v x y $.
$d x y $.

$hx $f wff x $.
$hy $f wff y $.

$a1 $a |- x $.

```

test71_empty_dollar_d.mm [REJECT] *Sec. 4.2.4: \ \$d requires at least two variables*

```

$( Unit Test 71: empty $d statement $)
$( Should reject: True $)
$( Book grammar: disjoint-stmt ::= '$d' variable variable variable* '$.'
requires at least two variables; metamath.exe errors; metamath-knife
errors ("Empty math string"). $)

$c wff $.
$d $.

```

test72_single_variable_dollar_d.mm [REJECT] *Sec. 4.2.4: \ \$d requires at least two variables*

```

$( Unit Test 72: single-variable $d statement $)

```

```
$( Should reject: True $)
$( Book 4.2.4: a simple $d has two different active variables; the
  appendix grammar requires at least two. metamath.exe errors;
  metamath-knife also rejects (nonzero exit), reporting it at warning
  severity: "A $d statement which lists only one variable is
  meaningless". $)

$c wff $.
$v x $.
$d x $.
```

test73_compound_dollar_d.mm [ACCEPT] *Sec. 4.2.4: compound \ \$d with three variables is legal*

```
$( Unit Test 73: compound $d with three variables $)
$( Should reject: False $)
$( Book 4.2.4: a compound $d has three or more variables, all different.
  Guards the arity gate against over-rejection. $)

$c wff $.
$v x y z $.
$d x y z $.
```

test76_unclosed_dollar_d.mm [REJECT] *Sec. 4.2.4: unterminated \ \$d is unclosed, not too-short*

```
$( Unit Test 76: unterminated $d at end of file $)
$( Should reject: True $)
$( Classification check: an unterminated $d must be reported as an
  unclosed statement, NOT as a too-short one -- the arity gate applies
  only to statements that actually reach their $. terminator. $)

$c wff $.
$v x $.
$d x
```

test81_inactive_variable_in_dollar_d.mm [REJECT] *Sec. 4.2.4: \ \$d requires ACTIVE variables, not merely declared ones*

```
$v y $.
${
$v x $.
$}
$d x y $.
```

test56_duplicate_d_normalization_bad.mm [REJECT] *Sec. 4.2.4: duplicate \ \$d is normalized, not treated as a separate restriction*

```
$( Test 56: Duplicate $d normalization - duplicates allowed; should ACCEPT per Sec 4.2.4 (union of DV pairs)
  ↪ $)

$c wff |- $.
$v x $.

$d x x $.
$d x x $.

f1 $f wff x $.
a1 $a |- x $.
p1 $p |- x $= f1 a1 $.
```

C.7 §4.2.5 The \$f and \$e Statements

test08_variable_used_outside_scope.mm [REJECT] *Sec. 4.2.5: variable must have active \ \$f*

```
$( Unit Test 8: Variable used outside scope $)
$( Should reject: True $)
```

```

$c wff |- $.
${
  $v x $.
  wf $f wff x $.
}$
bad $a |- x $.

```

test10_duplicate_labels.mm [REJECT] *Sec. 4.2.5: label tokens must be unique*

```

$( Unit Test 10: Duplicate labels $)
$( Should reject: True $)

$c wff |- $.
$v x $.
dup $f wff x $.
dup $a |- x $.

```

test11_label_conflicts_with_symbol.mm [REJECT] *Sec. 4.2.5: no label may match any math symbol*

```

$( Unit Test 11: Label conflicts with symbol $)
$( Should reject: True $)

$c wff |- $.
$v x $.
wff $f wff x $.

```

test12_non-constant_typecode.mm [REJECT] *Sec. 4.2.5: first symbol in \ \$f must be a constant typecode*

```

$( Unit Test 12: Non-constant typecode $)
$( Should reject: True $)

$c wff |- $.
$v x y $.
bad $f x y $.

```

test13_f_with_undeclared_variable.mm [REJECT] *Sec. 4.2.5: \ \$f variable must be declared*

```

$( Unit Test 13: $f with undeclared variable $)
$( Should reject: True $)

$c wff |- $.
bad $f wff undeclared $.

```

test14_variable_without_f_hypothesis.mm [REJECT] *Sec. 4.2.5: each variable in \ \$e/\ \$a/\ \$p must have active \ \$f*

```

$( Unit Test 14: Variable without $f hypothesis $)
$( Should reject: True $)

$c wff |- $.
$v x $.
bad $a |- x $.

```

test15_multiple_f_for_same_variable_bad.mm [REJECT] *Sec. 4.2.5: a variable may have only one active \ \$f statement*

```

$( Unit Test 15: Multiple $f for same variable $)
$( Should reject: True $)

$c wff class $.
$v x $.
f1 $f wff x $.
f2 $f class x $.

```

test16_conflicting_typecodes_bad.mm [REJECT] *Sec. 4.2.5: a variable may have only one*

active $\backslash$ \$f statement, including across nested blocks

```
$( Unit Test 16: Conflicting typecodes $)
$( Should reject: True $)

$c wff class $.
$v x $.
${
  f1 $f wff x $.
  ${
    f2 $f class x $.
  }
}$
```

test35_f_not_active_after_block_close.mm [REJECT] *Sec. 4.2.5: $\backslash$ \$f is active only until block end*

```
$( Unit Test 35: $f not active after block close $)
$( Should reject: True $)

$c wff |- $.
${
  $v x $.
  wf $f wff x $.
}$
$( x and wf are no longer active $)
bad $a |- x $.
```

test61_empty_expression.mm [ACCEPT] *Sec. 4.2.5: empty expression after typecode is valid*

```
$( test61_empty_expression.mm - ACCEPT
Tests spec 4.1.3 L153-154, L167-169:
"$e statement consists of ... zero or more active math symbols"
"$a statement consists of ... zero or more active math symbols"

Empty expressions (just typecode, no symbols) should be valid.
$)

$c wff |- $.

$( Axiom with empty expression after typecode $)
ax-empty $a wff $.

$( Essential hypothesis with empty expression $)
${
  $v p $.
  wp $f wff p $.

  $( Empty essential hypothesis $)
  he $e |- $.

  $( Proof that uses the empty e-hyp $)
  th1 $p wff p $= wp $.
}$
```

test64_multiple_ehyp_order.mm [ACCEPT] *Sec. 4.2.5, 4.2.6: mandatory hypothesis ordering*

```
$( test64_multiple_ehyp_order.mm - ACCEPT
Tests spec 4.1.4 L182-184, L211-214:
"The (possibly empty) set of mandatory hypotheses is the set of all
active $f statements containing mandatory variables, together with
all active $e statements."

"causes them to match the topmost entries of the stack, in order of
occurrence of the mandatory hypotheses"

Key insight: mandatory hypotheses = $f (for mandatory vars) + $e (all active)
in order of OCCURRENCE in the file.
```

```

$)

$c wff |- ( ) -> $.
$v p q $.
$wp $f wff p $.
$wq $f wff q $.

$( Build implication $)
$wi $a wff ( p -> q ) $.

$( A theorem with two essential hypotheses.
Mandatory hyps in occurrence order: wp, wq, e1, e2 $)
${
  e1 $e |- p $.
  e2 $e |- ( p -> q ) $.
  $( Modus ponens: from p and p->q, derive q $)
  mp $a |- q $.
}$

$( Use mp with concrete values.
To apply mp where p:=( p -> p ), q:=( p -> p ):
Push order: wp (for p), wp (for q), inst of e1, inst of e2

We'll prove: from ( p -> p ) and ( ( p -> p ) -> ( p -> p ) ), get ( p -> p )
$)

${
  h1 $e |- ( p -> p ) $.
  h2 $e |- ( ( p -> p ) -> ( p -> p ) ) $.

  $( Stack for mp: wp wp wi, wp wp wi, h1, h2
  This substitutes p:=( p -> p ), q:=( p -> p ) $)
  result $p |- ( p -> p ) $=
    wp wp wi wp wp wi h1 h2 mp $.
}$

```

test67_toplevel_essential.mm [ACCEPT] *Sec. 4.2.5, 4.2.8: top-level \ \$e is allowed (knife non-compliant)*

```

$( Unit Test 67: Top-level $e statements $)
$( Tests: Essential hypotheses at the outermost scope level
Spec allows this - no restriction in EBNF or spec text.
metamath-knife incorrectly REJECTS this (non-compliant).
Official Metamath correctly ACCEPTS this. $)
$( Should accept: True $)

$c wff |- $.
$v ph ps $.
$wp $f wff ph $.
$wq $f wff ps $.

$( Top-level essential hypothesis - not inside any ${ $} block $)
hyp1 $e |- ph $.

$( This assertion has hyp1 as a mandatory hypothesis $)
ax1 $a |- ps $.

```

C.8 §4.2.6 Assertions (\$a and \$p Statements)

missing-dollar-p.mm [REJECT] *Sec. 4.2.6: missing \ \$p statement*

```

${ mytoken $}

```

test21_self-referential_proof.mm [REJECT] *Sec. 4.2.6: proof may not reference itself*

```

$( Unit Test 21: Self-referential proof $)

```

```
$( Should reject: True $)

$c wff |- -> ( ) $.
$v x $.
 wf $f wff x $.
evil $p |- ( x -> x ) $= ( evil ) A $.
```

test22_typecode_mismatch_in_substitution.mm [REJECT] *Sec. 4.2.6: substitution must match typecode*

```
$( Unit Test 22: Typecode mismatch in substitution $)
$( Should reject: True $)

$c wff class |- $.
$v x y $.
f1 $f wff x $.
f2 $f class y $.
ax $a |- x $.
bad $p |- y $= f2 ax $.
```

test24_undefined_label_in_proof.mm [REJECT] *Sec. 4.2.6: proof labels must be defined*

```
$( Unit Test 24: Undefined label in proof $)
$( Should reject: True $)

$c wff |- $.
$v x $.
 wf $f wff x $.
ax $a |- x $.
bad $p |- x $= nosuchlabel $.
```

test26_wrong_conclusion_in_proof.mm [REJECT] *Sec. 4.2.6: final stack must match the \ \$p statement*

```
$( Unit Test 26: Wrong conclusion in proof $)
$( Should reject: True $)

$c wff |- $.
$v x y $.
f1 $f wff x $.
f2 $f wff y $.
ax $a |- x $.
bad $p |- y $= f1 ax $.
```

test32_forward_reference_in_proof.mm [REJECT] *Sec. 4.2.6: proof labels must be previously defined*

```
$( Unit Test 32: Forward reference in proof $)
$( Should reject: True $)

$c wff |- $.
$v x $.
 wf $f wff x $.
early $p |- x $= wf later $.
later $a |- x $.
```

test33_compressed_proof_stack_underflow.mm [REJECT] *Sec. 4.2.6: proof stack must not underflow*

```
$( Unit Test 33: Compressed proof stack underflow $)
$( Should reject: True $)

$c wff |- $.
$v x $.
 wf $f wff x $.
ax $a |- x $.
$( C tries to pop 3rd item, but stack has only 1 $)
```

```
bad $p |- x $= ( ax ) C $.
```

test62_self_substitution.mm [ACCEPT] *Sec. 4.2.6: a variable may map to an expression containing itself*

```
$( test62_self_substitution.mm - ACCEPT
  Tests spec 4.1.4 L204-205:
  "It is acceptable for a variable to be mapped to an expression
  containing it."

  This tests that p can be substituted with an expression containing p.
$)

$c wff |- ( ) -> $.
$v p q $.
$wp $f wff p $.
$wq $f wff q $.

$( Build implication: wff ( p -> q ) $)
$wi $a wff ( p -> q ) $.

$( Axiom: |- ( p -> p )
  Mandatory hypothesis: just wp (only var p is used) $)
$ax-id $a |- ( p -> p ) $.

$( Prove ( ( p -> p ) -> ( p -> p ) )
  We apply ax-id with p := ( p -> p )
  This means p maps to an expression containing p itself!

  ax-id needs: wp
  We need to push wff ( p -> p ) to substitute for p.
  Building wff ( p -> p ): wp wp wi (gives wff ( p -> p ))
  Then apply ax-id.
$)
$th-self $p |- ( ( p -> p ) -> ( p -> p ) ) $=
  wp wp wi ax-id $.
```

C.9 §4.2.8 Scoping Statements (\${ and \$})

test04_unbalanced_block_delimiters.mm [REJECT] *Sec. 4.2.8: \\${ must match \}*

```
$( Unit Test 4: Unbalanced block delimiters $)
$( Should reject: True $)

$c wff |- $.
${
  $v x $.
  wf $f wff x $.
$( missing $} $)
$ax $a |- x $.
```

test47_constant_inner_scope_direct.mm [REJECT] *Sec. 4.2.8: \ \$c must be in the outermost block*

```
$( Unit Test 47b: $c declaration directly in inner scope $)
$( Category: CORE - CRITICAL scoping test $)
$( Should reject: True - $c must be in outermost block $)
$( Spec Section 4.1.2 L1088: "All $c statements must be placed in the outermost block" $)

$c wff |- $.

${
  $( INVALID: $c declaration inside block - must be in outermost scope only $)
  $c inner-const $.

  $v x $.
}
```

```

wx $f wff x $.
ax-x $a |- x $.
$}

```

test75_dollar_d_nested_scope.mm [ACCEPT] *Sec. 4.2.8: $\backslash d$ is legal inside a nested block*

```

$( Unit Test 75: $d inside a nested block $)
$( Should reject: False $)
$( Sec. 4.2.8: $d is a normal statement and may occur in any scope. $)

$c wff $.
$v x y $.
${
  $d x y $.
$}

```

test60_empty_block.mm [ACCEPT] *Sec. 4.2.8: empty block $\backslash \{ \backslash \}$ is valid*

```

$( test60_empty_block.mm - ACCEPT
Tests EBNF: block ::= '{' stmt* '}'
Zero statements in a block is valid.

Empty blocks between valid proofs should be accepted.
$)

$c wff |- $.
$v p q $.
wp $f wff p $.
wq $f wff q $.

$( First proof $)
th1 $a wff p $.

$( Empty block - should be valid $)
${ $}

$( Another proof after empty block $)
th2 $a wff q $.

$( Nested empty blocks $)
${
  ${ $}
  ${ $}
$}

$( Final proof $)
th3 $p wff p $= wp $.

```

C.10 §4.4.5 Compressed Proof Format

out-of-range-saved-step-bad1.mm [REJECT] *Sec. 4.4.5 / Appendix B: compressed proof Z reference out of range*

```

$( out-of-range-saved-step-bad1.mm
Incorrect compressed proof: An example generated by ChatGPT o3
That references a proof index greater than the number of saved proof steps.
Added because mmverify.py was missing a raise MMErrror and tests did not flag it. $)

$c |- T $.

ax    $a |- T $.
buggy $p |- T $= ( ax ) A Z C $.

```

test19_illegal_characters_in_compressed_proof.mm [REJECT] *Sec. 4.4.5 / Appendix B: compressed proof uses only A-Z and ? (strict)*

```
$( Unit Test 19: Illegal characters in compressed proof $)
$( Should reject: True $)
```

```
$c wff |- $.
$v x $.
wf $f wff x $.
ax $a |- x $.
th $p |- x $= ( ax ) AOB $.
```

test29_compressed_proof_header_mismatch.mm [REJECT] *Sec. 4.4.5 / Appendix B: compressed proof header must match mandatory hyps*

```
$( Unit Test 29: Compressed proof header mismatch $)
$( Should reject: True $)
```

```
$c wff |- $.
$v x $.
wf $f wff x $.
ax1 $a |- x $.
ax2 $a |- x $.
$( Header lists ax1 but proof uses ax2! $)
bad $p |- x $= ( ax1 ) B $.
```

test36_whitespace_in_compressed_proof_bad.mm [REJECT] *Sec. 4.4.5 / Appendix B: no whitespace inside compressed proof string*

```
$( Unit Test 36: Whitespace in compressed proof $)
$( Should reject: True - This database has invalid proof steps B and C $)
$( Original intent was to test whitespace handling, but ABC is invalid $)
```

```
$c wff |- $.
$v x $.
wf $f wff x $.
ax $a |- x $.
$( Compressed proof ABC: A=0 (wf), B=1 (ax), C=2 (out of bounds!) $)
th $p |- x $= ( ax ) A
  B
  C $.
```

test38_whitespace_in_valid_compressed.mm [ACCEPT] *Sec. 4.4.5 / Appendix B: whitespace allowed in the header, not the proof string*

```
$( Unit Test 38: Whitespace in valid compressed proof $)
$( Should reject: False - whitespace should be ignored per spec section 4.4.2 $)
```

```
$c wff |- $.
$v x $.
wf $f wff x $.
ax $a |- x $.

$( Compressed proof with whitespace: spaces, newlines, tabs between valid steps $)
$( Proof is just: push wf (A), then ax (B) $)
$( Label list: [wf=0, ax=1], so A B is valid $)
th $p |- x $= ( ax ) A
  B
  $.
```

test97_illegal_character_mid_compressed_number.mm [REJECT] *Sec. 4.4.5 / Appendix B: illegal byte inside an unfinished U-Y compressed number*

```
$( Unit Test 97: Illegal character inside an unfinished U-Y compressed number $)
$( Should reject: True $)
```

```
$c wff |- $.
$v x $.
wf $f wff x $.
ax $a |- x $.
th $p |- x $= ( ax ) UOB $.
```

test98_compressed_save_interrupts_index_bad.mm [REJECT] *Sec. 4.4.5 / Appendix B: Z cannot interrupt an unfinished U-Y compressed number*

```
$c |- p $.

ax $a |- p $.

${
  left $e |- p $.
  right $e |- p $.
  merge $a |- p $.
}$

bad $p |- p $= ( ax merge ) AUZCB $.
```

test99_compressed_repeated_save_alias_bad.mm [REJECT] *Sec. 4.4.5 / Appendix B: each Z must immediately follow a completed proof step*

```
$c |- p $.

ax $a |- p $.

${
  left $e |- p $.
  right $e |- p $.
  merge $a |- p $.
}$

bad $p |- p $= ( ax merge ) AZZDB $.
```

test100_compressed_header_explicit_mandatory_hyp_bad.mm [REJECT] *Sec. 4.4.5 / Appendix B: the explicit compressed header may not repeat an implicit mandatory hypothesis*

```
$( Unit Test 100: A compressed-proof header must not repeat an implicit
mandatory hypothesis. Appendix B permits only non-mandatory hypotheses in
the parenthesized label list. If the duplicate were accepted, BC would use
the second wx heap entry and then apply ax, so the malformed proof would
otherwise verify successfully. $)
$( Should reject: True $)

$c wff |- $.
$v x $.

wx $f wff x $.
ax $a |- x $.

bad $p |- x $= ( wx ax ) BC $.
```

test55_compressed_z_oob.mm [REJECT] *Sec. 4.4.5 / Appendix B: Z reference out of bounds*

```
$( Test 55: Compressed proof index out of bounds - must REJECT per Appendix B (label index must be within
↪ header list) $)

$c wff |- $.
$v ph $.
f1 $f wff ph $.
a1 $a |- ph $.

p1 $p |- ph $= ( a1 ) B $.
```

test58_compressed_ehyp_z.mm [ACCEPT] *Sec. 4.4.5 / Appendix B: compressed proof with essential hypotheses and Z markers*

```
$( Minimal test: compressed proof with essential hypotheses AND Z marker $)
$( Based on demo0.mm structure - tests the combination that fails in big-unifier.mm $)
$(
  This test case is designed to isolate the bug where:
```

```

- Compressed proofs work (test57 passes)
- Normal proofs with e-hyps work (demo0.mm passes)
- BUT compressed proofs with e-hyps + Z marker FAIL (big-unifier.mm fails)

This file contains both normal and compressed versions of the same proof,
allowing direct comparison of verification behavior.
$)

$c 0 + = -> ( ) term wff |- $.
$v t r s P Q $.

tt $f term t $.
tr $f term r $.
ts $f term s $.
wp $f wff P $.
wq $f wff Q $.

$( Define term $)
tze $a term 0 $.
tpl $a term ( t + r ) $.

$( Define wff $)
weq $a wff t = r $.
wim $a wff ( P -> Q ) $.

$( Axioms $)
a1 $a |- ( t = r -> ( t = s -> r = s ) ) $.
a2 $a |- ( t + 0 ) = t $.

$( Modus ponens - has essential hypotheses $)
${
  min $e |- P $.
  maj $e |- ( P -> Q ) $.
  mp $a |- Q $.
}$

$( Normal proof of |- t = t - should work $)
th1_normal $p |- t = t $=
  tt tze tpl tt weq tt tt weq tt a2 tt tze tpl tt weq tt tze tpl tt weq tt tt
  weq wim tt a2 tt tze tpl tt tt a1 mp mp $.

$(
  Compressed proof of |- t = t generated by Metamath program.
  This proof:
  1. Uses compressed format
  2. Uses modus ponens (mp) which has essential hypotheses
  3. Uses Z marker to save/recall intermediate results

  Labels:
    A=tt (mandatory f-hyp)
    B=tze, C=tpl, D=weq, E=a2, F=wim, G=a1, H=mp (explicit labels)

  Proof string: ABCZADZAADZAEZJJKFLIAAGHH

  The Z at position 4 saves something, then J (index 9) recalls it.
  This combines all three ingredients that cause big-unifier.mm to fail.
$)
th1_compressed $p |- t = t $=
  ( tze tpl weq a2 wim a1 mp ) ABCZADZAADZAEZJJKFLIAAGHH $.

```

test65_compressed_assertion_fhyp_order.mm [ACCEPT] *Sec. 4.4.5 / Appendix B: compressed proofs with scoped f-hyps*

```

$( Unit Test 65: Compressed proof with 3-variable axiom $)
$( Tests: Correctly using an axiom with 3 f-hyps in a compressed proof.
  ax-3var has mandatory hyps: wff ph, wff ps, wff ch, |- ph, |- ps
  We prove |- ps by calling ax-3var with substitution ch -> ps
  IMPORTANT: Everything wrapped in outer ${ $} to avoid top-level $e $)

```

```

$( Should accept: True $)

$c wff |- $.
$v ph ps ch $.

${
  wp $f wff ph $.
  wq $f wff ps $.
  wc $f wff ch $.

  $( Axiom: from |- ph and |- ps, conclude |- ch $)
  ax.1 $e |- ph $.
  ax.2 $e |- ps $.
  ax-3var $a |- ch $.

  $( Theorem: prove |- ps by using ax-3var with ch -> ps $)
  $( Mandatory hyps of th: wp(A), wq(B), ax.1(C), ax.2(D), th.1(E), th.2(F)
    Note: ax.1 and ax.2 are mandatory because they're used in the proof!
    Label list: (ax-3var) -> G = ax-3var
    Proof: A B B E F G = push wff ph, wff ps, wff ps, |- ph, |- ps, call ax-3var
    ax-3var with {ph->ph, ps->ps, ch->ps} yields |- ps $)
  th.1 $e |- ph $.
  th.2 $e |- ps $.
  th $p |- ps $= ( ax-3var ) ABBEFG $.
}

```

test66.compressed_mand_hyp_order.mm [ACCEPT] *Sec. 4.4.5 / Appendix B: mandatory hypothesis ordering*

```

$( Unit Test 66: Compressed proof - mandatory hypothesis ordering $)
$( Tests: Mandatory hyps are f-hyps (in order) + e-hyps (in order) $)
$( Should accept: True $)

$c wff |- $.
$v ph ps $.
wp $f wff ph $.
wq $f wff ps $.

${
  $( Axiom with 2 essential hyps $)
  min.1 $e |- ph $.
  min.2 $e |- ps $.
  ax-min $a |- ph $.
}

${
  $( Proof using compressed format $)
  $( Mandatory hyps: wp(A), wq(B), th.1(C), th.2(D). Label list: E=ax-min $)
  th.1 $e |- ph $.
  th.2 $e |- ps $.
  th $p |- ph $= ( ax-min ) ABCDE $.
}

```

test56.compressed_z_simple_bad.mm [REJECT] *Sec. 4.4.5 / Appendix B: compressed Z reference out of bounds*

```

$( Minimal test of compressed proof with Z (save) marker $)

$c wff |- $.
$v ph ps $.
wph $f wff ph $.
wps $f wff ps $.

$( A simple axiom - implication introduction $)
ax-1 $a |- ( ph -> ( ps -> ph ) ) $.

$( Axiom using ax-1 as basis $)

```

```

wi $a wff ( ph -> ps ) $.

$( Prove something using Z - save and reuse $)
$( This proof:
- A = index 0 = wph (mandatory f-hyp for ph)
- B = index 1 = wps (mandatory f-hyp for ps)
- C = index 2 = wi (explicit label)
- D = index 3 = ax-1 (explicit label)
So: ( wi ax-1 ) A B C Z D means:
1. Push wph -> get "wff ph" on stack
2. Push wps -> get "wff ps" on stack
3. Apply wi -> get "wff ( ph -> ps )" on stack
4. Z = save top (which is "wff ( ph -> ps )")
5. Apply ax-1 with substitutions -> get "|- ( ph -> ( ps -> ph ) )"
Actually, let me simplify...
$)

$( Simple proof without Z first to verify basic compressed proofs work $)
simple1 $p wff ( ph -> ps ) $= ( wi ) AB $.

$( Now with Z $)
$.

```

C.11 §4.4.6 Unknown Proofs or Subproofs

test30_qmark_in_compressed_proof.mm [ACCEPT] *Sec. 4.4.6 / Appendix B: ? allowed in compressed proofs*

```

$( Unit Test 30: ? in compressed proof $)
$( Should reject: False $)

$c wff |- $.
$v x $.
wf $f wff x $.
ax $a |- x $.
incomplete $p |- x $= ( ax ) ? $.

```

C.12 Appendix E: Metamath Language EBNF

test77_empty_constant_statement.mm [REJECT] *App. E: \ \$c requires at least one constant*

```

$( Unit Test 77: $c statement with no constants $)
$( Should reject: True $)
$( Appendix E grammar: constant-stmt ::= '$c' constant+ '$.' requires at
least one constant. metamath.exe and metamath-knife both reject. $)

$c a $.
$c $.

```

test78_empty_variable_statement.mm [REJECT] *App. E: \ \$v requires at least one variable*

```

$( Unit Test 78: $v statement with no variables $)
$( Should reject: True $)
$( Appendix E grammar: variable-stmt ::= '$v' variable+ '$.' requires at
least one variable. metamath.exe and metamath-knife both reject. $)

$v $.

```

test79_multi_constant_and_variable.mm [ACCEPT] *App. E: multi-symbol \ \$c / \$v remain legal*

```

$( Unit Test 79: multi-symbol $c and $v remain legal $)
$( Should reject: False $)
$( Guards the arity gates against over-rejection. $)

$c a b c $.

```

```
$v x y z $.
```

C.13 Proof Verification Failures (§4.1.4)

anatomy-bad1.mm [REJECT] 4.2.1: proof verification failure

```
$( anatomy.mm $)

$(
    PUBLIC DOMAIN DEDICATION

    This file is placed in the public domain per the Creative Commons Public
    Domain Dedication. http://creativecommons.org/licenses/publicdomain/

    Norman Megill - email: nm at alum.mit.edu

$)

$( This file is the introductory formal system example described
    in Chapter 4 of the Metamath book, "Anatomy of a proof". $)

$( Declare the constant symbols we will use $)
$c ( ) -> wff $.
$( Declare the metavariables we will use $)
$v p q r s $.
$( Declare wffs $)
wp $f wff p $.
wq $f wff q $.
wr $f wff r $.
ws $f wff s $.
w2 $a wff ( p -> q ) $.

$( Prove a simple theorem. $)
nnew $p wff ( s -> ( r -> p ) ) $= ws wr wp w2 ws $.
```

anatomy-bad2.mm [REJECT] 4.2.1: proof verification failure

```
$( anatomy.mm $)

$(
    PUBLIC DOMAIN DEDICATION

    This file is placed in the public domain per the Creative Commons Public
    Domain Dedication. http://creativecommons.org/licenses/publicdomain/

    Norman Megill - email: nm at alum.mit.edu

$)

$( This file is the introductory formal system example described
    in Chapter 4 of the Metamath book, "Anatomy of a proof". $)

$( Declare the constant symbols we will use $)
$c ( ) -> wff $.
$( Declare the metavariables we will use $)
$v p q r s $.
$( Declare wffs $)
wp $f wff p $.
wq $f wff q $.
wr $f wff r $.
ws $f wff s $.
w2 $a wff ( p -> q ) $.

$( Prove a simple theorem. $)
nnew $p wff ( s -> ( r -> p ) ) $= ws wr wp w2 $.
```

anatomy-bad3.mm [REJECT] 4.2.1: proof verification failure

```
$( anatomy.mm $)

$(
    PUBLIC DOMAIN DEDICATION

    This file is placed in the public domain per the Creative Commons Public
    Domain Dedication. http://creativecommons.org/licenses/publicdomain/

    Norman Megill - email: nm at alum.mit.edu

$)

$( This file is the introductory formal system example described
    in Chapter 4 of the Metamath book, "Anatomy of a proof". $)

$( Declare the constant symbols we will use $)
$c ( ) -> wff $.
$( Declare the metavariables we will use $)
$v p q r s $.
$( Declare wffs $)
wp $f wff p $.
wq $f wff q $.
wr $f wff r $.
ws $f wff s $.
w2 $a wff ( p -> q ) $.

$( Prove a simple theorem. $)
nnew $p wff ( s -> ( r -> p ) ) $= wr wp w2 w2 $.
```

demo0-bad1.mm [REJECT] 4.2.1: proof verification failure

```
$( demo0.mm 1-Jan-04 $)

$(
    PUBLIC DOMAIN DEDICATION

    This file is placed in the public domain per the Creative Commons Public
    Domain Dedication. http://creativecommons.org/licenses/publicdomain/

    Norman Megill - email: nm at alum.mit.edu

$)

$( This file is the introductory formal system example described
    in Chapter 2 of the Metamath book. $)

$( Declare the constant symbols we will use $)
$c 0 + = -> ( ) term wff |- $.
$( Declare the metavariables we will use $)
$v t r s P Q $.
$( Specify properties of the metavariables $)
tt $f term t $.
tr $f term r $.
ts $f term s $.
wp $f wff P $.
wq $f wff Q $.
$( Define "term" (part 1) $)
tze $a term 0 $.
$( Define "term" (part 2) $)
tpl $a term ( t + r ) $.
$( Define "wff" (part 1) $)
weq $a wff t = r $.
$( Define "wff" (part 2) $)
```

```

wim $a wff ( P -> Q ) $.
$( State axiom a1 $)
a1 $a |- ( t = r -> ( t = s -> r = s ) ) $.
$( State axiom a2 $)
a2 $a |- ( t + 0 ) = t $.
${
  min $e |- P $.
  maj $e |- ( P -> Q ) $.
$( Define the modus ponens inference rule $)
  mp $a |- Q $.
$}
$( Prove a theorem $)
th1 $p |- t = t $=
$( Here is its proof: $)
  tt tze tt tpl weq tt tt weq tt a2 tt tze tpl
  tt weq tt tze tpl tt weq tt tt weq wim tt a2
  tt tze tpl tt tt a1 mp mp
$.

```

issue107.mm [REJECT] 4.2.1: proof verification failure (missing hypothesis)

```

$c axiom hypothesis thesis $.

ax $a axiom $.

${
  hyp $e hypothesis $.
  mp $a thesis $.
$}

th $p thesis $= ax mp $.

```

C.14 Active Hypotheses (§4.1.4)

test23_using_non-essential_hypotheses.mm [REJECT] 4.2.5: only active hypotheses may be used

```

$( Unit Test 23: Using non-essential hypotheses $)
$( Should reject: True $)

$c wff |- $.
$v x $.
wf $f wff x $.
${
  hyp $e |- x $.
  th1 $p |- x $= hyp $.
$}
evil $p |- x $= ( hyp ) A $.

```

C.15 Disjoint Variable Violations (§4.1.4)

disjoint1.mm [REJECT] 4.2.7: \ \$d violation in proof

```

$c var formula |- $.
$v x y $.
xf $f formula x $.
yf $f formula y $.
${
  $d x y $.
  ax-1 $a |- x y $.
$}
bad $p |- x x $=
xf xf ax-1 $.

```

disjoint2.mm [REJECT] 4.2.7: \ \$d violation in proof

```

$c formula |- ( ) $.

$v x y z w $.
$xf $f formula x $.
$yf $f formula y $.
$zf $f formula z $.
$wf $f formula w $.
$combo $a formula ( x y ) $.
${
  $d x y $.
  $ax-1 $a |- ( x y ) $.
$}

$verybad $p |- ( ( x y ) ( z x ) ) $=
$xf $yf $combo $zf $xf $combo $ax-1 $.

${
  $d x y z $.
  $stillbad $p |- ( ( x y ) ( z x ) ) $=
  $xf $yf $combo $zf $xf $combo $ax-1 $.
$}

$semigood $p |- ( ( x y ) ( z w ) ) $=
$xf $yf $combo $zf $wf $combo $ax-1 $.

${
  $d x y $.
  $d z w $.
  $almostgood $p |- ( ( x y ) ( z w ) ) $=
  $xf $yf $combo $zf $wf $combo $ax-1 $.
$}

${
  $d x y z w $.
  $good $p |- ( ( x y ) ( z w ) ) $=
  $xf $yf $combo $zf $wf $combo $ax-1 $.
$}

${
  $d x z $. $d x w $. $d y z $. $d y w $.
  $stillgood $p |- ( ( x y ) ( z w ) ) $=
  $xf $yf $combo $zf $wf $combo $ax-1 $.
$}

```

disjoint3.mm [REJECT] 4.2.7: \ \$d violation in proof

```

$c var formula |- $.
$v x y $.
$xf $f formula x $.
$yf $f formula y $.
${
  $d x y $.
  $ax-1 $a |- x y $.
$}
$bad $p |- x y $=
$xf $yf $ax-1 $.

```

test27_disjoint_variable_constraint_violation.mm [REJECT] 4.2.7: \ \$d constraints must be satisfied

```

$( Unit Test 27: Disjoint variable constraint violation $)
$( Should reject: True $)

$c wff -> |- ( ) $.
$v x y z $.
$wfx $f wff x $.
$wfy $f wff y $.

```

```

wfz $f wff z $.
$d x y $.

$( Axiom mentions BOTH x and y, so $d x y is mandatory for it $)
axxy $a |- ( x -> y ) $.

$( This proof violates $d: substitutes z for both x and y $)
$( x:=z, y:=z violates $d x y $)
bad $p |- ( z -> z ) $= wfz wfz axxy $.

```

C.16 Valid Databases (Positive Tests)

anatomy.mm [ACCEPT] *Valid database per spec*

```

$( anatomy.mm $)

$(
    PUBLIC DOMAIN DEDICATION

    This file is placed in the public domain per the Creative Commons Public
    Domain Dedication. http://creativecommons.org/licenses/publicdomain/

    Norman Megill - email: nm at alum.mit.edu

$)

$( This file is the introductory formal system example described
    in Chapter 4 of the Metamath book, "Anatomy of a proof". $)

$( Declare the constant symbols we will use $)
$c ( ) -> wff $.
$( Declare the metavariables we will use $)
$v p q r s $.
$( Declare wffs $)
wp $f wff p $.
wq $f wff q $.
wr $f wff r $.
ws $f wff s $.
w2 $a wff ( p -> q ) $.

$( Prove a simple theorem. $)
nnew $p wff ( s -> ( r -> p ) ) $= ws wr wp w2 w2 $.

```

demo0-includee.mm [ACCEPT] *Valid database per spec*

```

$( demo0-includee.mm 1-Jan-04 $)

$(
    PUBLIC DOMAIN DEDICATION

    This file is placed in the public domain per the Creative Commons Public
    Domain Dedication. http://creativecommons.org/licenses/publicdomain/

    Norman Megill - email: nm at alum.mit.edu

$)

$( This file is the introductory formal system example described
    in Chapter 2 of the Meamath book. $)

$( Declare the constant symbols we will use $)
$c 0 + = -> ( ) term wff |- $.
$( Declare the metavariables we will use $)
$v t r s P Q $.
$( Specify properties of the metavariables $)

```

```

tt $f term t $.
tr $f term r $.
ts $f term s $.
wp $f wff P $.
wq $f wff Q $.
$( Define "term" (part 1) $)
tze $a term 0 $.
$( Define "term" (part 2) $)
tpl $a term ( t + r ) $.
$( Define "wff" (part 1) $)
weq $a wff t = r $.
$( Define "wff" (part 2) $)
wim $a wff ( P -> Q ) $.
$( State axiom a1 $)
a1 $a |- ( t = r -> ( t = s -> r = s ) ) $.
$( State axiom a2 $)
a2 $a |- ( t + 0 ) = t $.
${
  min $e |- P $.
  maj $e |- ( P -> Q ) $.
$( Define the modus ponens inference rule $)
  mp $a |- Q $.
$}

```

demo0-includer.mm [ACCEPT] *Valid database per spec*

```

$( demo0-includer.mm 1-Jan-04 $)

$(
  PUBLIC DOMAIN DEDICATION

  This file is placed in the public domain per the Creative Commons Public
  Domain Dedication. http://creativecommons.org/licenses/publicdomain/

  Norman Megill - email: nm at alum.mit.edu

$)

$[ demo0-includee.mm $]

$( Prove a theorem $)
th1 $p |- t = t $=
$( Here is its proof: $)
  tt tze tpl tt weq tt tt weq tt a2 tt tze tpl
  tt weq tt tze tpl tt weq tt tt weq wim tt a2
  tt tze tpl tt tt a1 mp mp
$.

```

demo0.mm [ACCEPT] *Valid database per spec*

```

$( demo0.mm 1-Jan-04 $)

$(
  PUBLIC DOMAIN DEDICATION

  This file is placed in the public domain per the Creative Commons Public
  Domain Dedication. http://creativecommons.org/licenses/publicdomain/

  Norman Megill - email: nm at alum.mit.edu

$)

$( This file is the introductory formal system example described
  in Chapter 2 of the Meamath book. $)

$( Declare the constant symbols we will use $)

```

```

$c 0 + = -> ( ) term wff |- $.
$( Declare the metavariables we will use $)
$v t r s P Q $.
$( Specify properties of the metavariables $)
tt $f term t $.
tr $f term r $.
ts $f term s $.
wp $f wff P $.
wq $f wff Q $.
$( Define "term" (part 1) $)
tze $a term 0 $.
$( Define "term" (part 2) $)
tpl $a term ( t + r ) $.
$( Define "wff" (part 1) $)
weq $a wff t = r $.
$( Define "wff" (part 2) $)
wim $a wff ( P -> Q ) $.
$( State axiom a1 $)
a1 $a |- ( t = r -> ( t = s -> r = s ) ) $.
$( State axiom a2 $)
a2 $a |- ( t + 0 ) = t $.
${
  min $e |- P $.
  maj $e |- ( P -> Q ) $.
$( Define the modus ponens inference rule $)
  mp $a |- Q $.
$}
$( Prove a theorem $)
th1 $p |- t = t $=
$( Here is its proof: $)
  tt tze tpl tt weq tt tt weq tt a2 tt tze tpl
  tt weq tt tze tpl tt weq tt tt weq wim tt a2
  tt tze tpl tt tt a1 mp mp
$.

```

empty.mm [ACCEPT] *Empty database is valid*

issue129.mm [ACCEPT] *Valid database per spec*

```

$c A $.

ax-1 $a A A A A A $.

th1 $p A A A A A $= ax-1 $.

$( $t
  latexdef "A" as "\text{A A A A A A A A }";
$)

```

issue134.mm [ACCEPT] *Valid database per spec*

```

$c A $.
ax1 $a A $.

```

local-var-good.mm [ACCEPT] *Valid variable scope per spec*

```

$c formula $.
$c A. $.
$c setvar $.
$v ph $.
wph $f formula ph $.

${
  $v x $.
  $( Let ' x ' be an individual variable (temporary declaration). $)
  vx.wal $f setvar x $.
  $( Extend formula definition to include the universal quantifier ('for all'). $)
  wal $a formula A. x ph $.
}

```

```
$}
```

min_found.mm [ACCEPT] *Valid database per spec*

```
$( A shortening found by the minimizer. The comparison with min_not_found.mm
  suggests that the minimizer's ability to detect it depends on the presence
  of statement "A" in the steps of the minimized proof. The constant C and
  axiom ax-3 are not used here, but they are kept anyway to lessen the amount
  of differences with min_not_found.mm. $)

$c A B C D $.

ax-1 $a A $.
ax-2 $a B $.
ax-3 $a C $.

${
  in1.1 $e A $.
  in1 $a D $.
}$

${
  in2.1 $e B $.
  in2.2 $e A $.
  in2.3 $e B $.
  in2.4 $e B $.
  in2.5 $e B $.
  in2 $a D $.
}$

$( Minimized proof, found automatically. (New usage is discouraged.) $)
th1 $p D $= ( ax-1 in1 ) AB $.

$( Non-minimized proof. $)
th2 $p D $= ( ax-2 ax-1 in2 ) ABAAAC $.
```

min_not_found.mm [ACCEPT] *Valid database per spec*

```
$( A shortening not found by the minimizer. The comparison with min_found.mm
  suggests that the minimizer's inability to detect it is due to the absence
  of statement "C" in any of the steps of the minimized proof. The
  shortening is detected if proveFloating is set to 1 in replaceStatement
  called by minimizeProof, which allows absent $e statements to be found. $)

$c A B C D $.

ax-1 $a A $.
ax-2 $a B $.
ax-3 $a C $.

${
  in1.1 $e A $.
  in1 $a D $.
}$

${
  in2.1 $e B $.
  in2.2 $e C $.
  in2.3 $e B $.
  in2.4 $e B $.
  in2.5 $e B $.
  in2 $a D $.
}$

$( Minimized proof, not found automatically. (New usage is discouraged.) $)
th1 $p D $= ( ax-1 in1 ) AB $.

$( Non-minimized proof. $)
```

```
th2 $p D $= ( ax-2 ax-3 in2 ) ABAAAC $.
```

nuniq-gram.mm [ACCEPT] *Valid database per spec (non-unique grammar allowed)*

```
$( Here we have two variable typecodes A and B, and one logical
   typecode |- , and the question is whether |- connects to A or B.
   It parses either way. This is actually an unambiguous formal system,
   because we require only that there are no two ways to derive the proof
   of the *same* syntax theorem, and "A *" and "B *" are different theorems. $)
$c A B |- * $. $v a b $.
va $f A a $.
vb $f B b $.
as $a A * $.
bs $a B * $.
ax $a |- * $.
```

test37_rpn_interleaving_mandatory_hyps.mm [ACCEPT] *Valid per RPN proof semantics*

```
$( Unit Test 37: RPN interleaving of $f and $e in mandatory hypotheses $)
$( Should reject: False - This documents correct behavior $)

$c wff |- -> ( ) $.
$v ph ps $.

${
  $( $f for ph $)
  wph $f wff ph $.

  $( $e using ph $)
  h1 $e |- ph $.

  $( $f for ps - AFTER the $e (interleaved!) $)
  wps $f wff ps $.

  $( Assertion: mandatory hyps in RPN order MUST be: wph, h1, wps $)
  $( NOT: wph, wps, h1 (which would be "all $f then $e") $)
  ax-test $a |- ( ph -> ps ) $.
$}

$( This test documents the CORRECT mandatory hypothesis order. $)
$( A verifier that incorrectly orders as "all $f, then all $e" would get: $)
$( WRONG: wph, wps, h1 $)
$( The correct order per spec is appearance order: $)
$( CORRECT: wph, h1, wps $)

$( To verify: use metamath.exe "show statement ax-test /full" $)
$( Expected output: "Its mandatory hypotheses in RPN order are:" $)
$( wph $f wff ph $. $)
$( h1 $e |- ph $. $)
$( wps $f wff ps $. $)
```

test39_include_basic_outer.mm [ACCEPT] *Valid include at outermost scope*

```
$( Unit Test 39: Basic include - outer file $)
$( Should reject: False - legitimate include should work $)

$( Include provides constants, variables, and axioms $)
$[ ./helpers/test39_helper.mm $]

$( Use the included axiom to prove something $)
th1 $p |- ph $= wph ax-1 $.
```

test_complex_ehyp_syl2anc.mm [ACCEPT] *Valid proof per spec*

```
$( Minimized test case for syl2anc - tests complex essential hypothesis handling $)
$( Based on hol.mm by Mario Carneiro $)
$( This proof SHOULD verify correctly $)

$c term |- |= ( ) , $.
```

```

$V R S T A B $.

tr $f term R $.
ts $f term S $.
tt $f term T $.
ta $f term A $.
tb $f term B $.

$( Syntax: context pair $)
kct $a term ( A , B ) $.

$( Axioms with essential hypotheses $)
${
  ax-syl.1 $e |- R |= S $.
  ax-syl.2 $e |- S |= T $.
  ax-syl $a |- R |= T $.
}$

${
  ax-jca.1 $e |- R |= S $.
  ax-jca.2 $e |- R |= T $.
  ax-jca $a |- R |= ( S , T ) $.
}$

$( Derived theorems $)
${
  syl.1 $e |- R |= S $.
  syl.2 $e |- S |= T $.
  syl $p |- R |= T $=
    tr ts tt syl.1 syl.2 ax-syl $.
}$

${
  jca.1 $e |- R |= S $.
  jca.2 $e |- R |= T $.
  jca $p |- R |= ( S , T ) $=
    tr ts tt jca.1 jca.2 ax-jca $.
}$

${
  syl2anc.1 $e |- R |= S $.
  syl2anc.2 $e |- R |= T $.
  syl2anc.3 $e |- ( S , T ) |= A $.
  syl2anc $p |- R |= A $=
    tr ts tt kct ta tr ts tt syl2anc.1 syl2anc.2 jca syl2anc.3 syl $.
}$

```

test59.f.type.global.conflict.mm [ACCEPT] *SPEC DIVERGENCE: Sec. 4.2.5 says REJECT, all impls ACCEPT*

```

$( test59.f.type.global.conflict.mm - ACCEPT (SPEC DIVERGENCE!)
Spec 4.1.3 L156-158 says:
"The type declared by a $f statement for a given label is global even if
the variable is not (e.g., a database may not have wff P in one local
scope and class P in another)."

HOWEVER: All three verifiers (metamath.exe, metamath-knife, mmverify.py)
ACCEPT having the same variable with different types in different scopes!

This is documented as a SPEC DIVERGENCE: the spec says REJECT, but
all implementations ACCEPT. The convention in set.mm is to never do this,
so it's never been caught.

For our test suite, we follow the IMPLEMENTATIONS (ACCEPT), not the spec.
$)

$c wff class |- $.

```

```

$( First scope: x is wff $)
${
  $v x $.
  wx $f wff x $.
  ax1 $a wff x $.
}$

$( Second scope: x is class - spec says REJECT, implementations say ACCEPT $)
${
  $v x $.
  cx $f class x $.
  ax2 $a class x $.
}$

```

test57_compressed_z_minimal.mm [ACCEPT] Valid compressed proof with Z

```

$( Minimal test of compressed proof with Z (save) marker $)
$( This is designed to be the simplest possible Z test that should pass $)

$c wff ( ) -> $.
$v ph ps $.
wph $f wff ph $.
wps $f wff ps $.

$( Binary connective - implication $)
wi $a wff ( ph -> ps ) $.

$( Simple proof using Z:
  Prove: wff ( ( ph -> ps ) -> ( ph -> ps ) )

  Labels: wph=0, wps=1, wi=2

  Proof steps:
  A = 0 = push wph -> stack: [wff ph]
  B = 1 = push wps -> stack: [wff ph, wff ps]
  C = 2 = apply wi -> stack: [wff ( ph -> ps )]
  Z = -1 = save top -> saved: [wff ( ph -> ps )], stack: [wff ( ph -> ps )]
  D = 3 = recall saved[0] -> stack: [wff ( ph -> ps ), wff ( ph -> ps )]
  C = 2 = apply wi -> stack: [wff ( ( ph -> ps ) -> ( ph -> ps ) )]

  Compressed: ABCZDC
$)
th1 $p wff ( ( ph -> ps ) -> ( ph -> ps ) ) $= ( wi ) ABCZDC $.

```

test_compressed_simple.mm [ACCEPT] Valid compressed proof

```

$( Minimal test for compressed proof $)

$c term |- |= $.
$v R S T $.

tR $f term R $.
tS $f term S $.
tT $f term T $.

${
  syl.1 $e |- R |= S $.
  syl.2 $e |- S |= T $.
  ax-syl $a |- R |= T $.
}$

${
  th.1 $e |- R |= S $.
  th.2 $e |- S |= T $.
  $( Uncompressed proof - should work $)
  th $p |- R |= T $= tR tS tT th.1 th.2 ax-syl $.
}$

```

test_compressed_syl.mm [ACCEPT] *Valid compressed proof: A-E push the five mandatory hypotheses and F applies ax-syl*

```
$( Minimal test for compressed proof with multiple floating hypotheses $)

$c term |- |= $.
$v R S T $.

tR $f term R $.
tS $f term S $.
tT $f term T $.

${
  syl.1 $e |- R |= S $.
  syl.2 $e |- S |= T $.
  $( Syllogism axiom $)
  ax-syl $a |- R |= T $.
}$

${
  th.1 $e |- R |= S $.
  th.2 $e |- S |= T $.
  $( Theorem using compressed proof - should push tR, tS, tT before ax-syl $)
  th $p |- R |= T $= ( ax-syl ) ABCDEF $.
}$
```

test_dv_pairwise.mm [ACCEPT] *Valid pairwise \ \$d*

```
$( Test for DV pairwise expansion: $d x y z $. should create pairs (x,y), (x,z), (y,z) $)

$c wff |- $.
$v x y z $.

wx $f wff x $.
wy $f wff y $.
wz $f wff z $.

${
  $d x y z $.

  $( Axiom requiring x,y disjoint $)
  ax-xy.1 $e |- x $.
  ax-xy $a |- y $.
}$

${
  $d x y z $.

  $( Axiom requiring x,z disjoint $)
  ax-xz.1 $e |- x $.
  ax-xz $a |- z $.
}$

${
  $d x y z $.

  $( Axiom requiring y,z disjoint - THIS IS THE KEY TEST $)
  ax-yz.1 $e |- y $.
  ax-yz $a |- z $.
}$

$( Test theorem - uses all three DV pairs $)
${
  $d x y z $.
  th.1 $e |- x $.
  th $p |- z $= wx wz th.1 ax-xz $.
}$
```

test_dv_yz_required.mm [ACCEPT] Valid \ \$d test

```
$( Test for DV pairwise expansion: $d x y z $. should create pair (y,z)
   This test REQUIRES (y,z) to be disjoint - will FAIL if pairwise expansion broken $)

$c wff |- $.
$v x y z $.

wx $f wff x $.
wy $f wff y $.
wz $f wff z $.

$( Key axiom: requires y and z to be disjoint $)
${
  $d y z $.
  ax-yz.1 $e |- y $.
  ax-yz $a |- z $.
}$}

$( Theorem using multi-variable $d declaration
   The $d x y z $. MUST expand to include (y,z) for this to verify! $)
${
  $d x y z $.
  th.1 $e |- y $.
  th $p |- z $= wy wz th.1 ax-yz $.
}$}
```

test_stack_fhyps.mm [ACCEPT] Valid stack test

```
$( Test for floating hypothesis consumption from stack $)

$c wff |- term |= $.
$v R S T $.

wR $f term R $.
wS $f term S $.
wT $f term T $.

${
  ax-syl.1 $e |- R |= S $.
  ax-syl.2 $e |- S |= T $.
  $( Syllogism: if R implies S and S implies T, then R implies T $)
  ax-syl $a |- R |= T $.
}$}

${
  syl.1 $e |- R |= S $.
  syl.2 $e |- S |= T $.
  $( Proof using ax-syl - should need 5 items on stack:
     3 floating hyps (term R, term S, term T) + 2 essential hyps $)
  syl $p |- R |= T $= wR wS wT syl.1 syl.2 ax-syl $.
}$}
```

test_stack_simple.mm [ACCEPT] Valid stack test

```
$( Simple test for stack operations $)

$c wff |- $.
$v x y $.

wx $f wff x $.
wy $f wff y $.

${
  ax $a |- x $.
  th $p |- y $= wy ax $.
}$}
```

C.17 Miscellaneous

test41_include_multiple_main.mm [ACCEPT] *Multiple valid includes at outermost scope*

```
$( Unit Test 41: Multiple includes $)
$( Should reject: False - multiple includes should work $)

$c wff |- $.

$( Include file A $)
$[ ./helpers/test41_helper_a.mm $]

$( Include file B $)
$[ ./helpers/test41_helper_b.mm $]

$( Use content from both includes $)
th-a $p |- var-a $= fa ax-a $.
th-b $p |- var-b $= fb ax-b $.
```

test_axmp_z.mm [REJECT] *Broken compressed proof - both verifiers reject*

```
$( Minimal test: ax-mp with essential hypotheses and Z marker $)

$c wff |- e ( , ) $.
$v x y $.

wx $f wff x $.
wy $f wff y $.

$( Binary connective $)
wi $a wff e ( x , y ) $.

$( Modus ponens with essential hypotheses $)
${
  ax-mp.1 $e |- x $.
  ax-mp.2 $e |- e ( x , y ) $.
  ax-mp $a |- y $.
}$

$( A simple axiom to use $)
ax1 $a |- e ( x , e ( y , x ) ) $.

$(
  Compressed proof using Z:
  Goal: |- e ( x , e ( y , x ) )

  Labels: wx=0, wy=1, wi=2, ax-mp=3, ax1=4

  Proof steps for normal proof:
  wx wy ax1 => |- e ( x , e ( y , x ) )

  As compressed: ( wi ax-mp ax1 ) ABC
  A=0=wx, B=1=wy, C=2=ax1
$)
th1 $p |- e ( x , e ( y , x ) ) $= ( wi ax-mp ax1 ) ABC $.
```

test_dv_multi.mm [REJECT] *Comment says INVALID PROOF - should be rejected*

```
$( Test for multi-variable $d declarations - INVALID PROOF (should be rejected) $)

$c wff |- $.
$v x y z $.

wx $f wff x $.
wy $f wff y $.
wz $f wff z $.

${
```

```

$d x y z $.
ax $a |- x $.

${
  th.1 $e |- y $.
  th $p |- z $= wy wz ax th.1 $.
}$
$}

```

C.18 Large Test Files (Summary Only)

The following test files exceed 70 lines and are summarized rather than listed verbatim.

File	Verdict	Lines	Spec Rule
big-unifier-bad1.mm	REJECT	100	4.2.1: unification/substitution failure
big-unifier-bad2.mm	REJECT	100	4.2.1: unification/substitution failure
big-unifier-bad3.mm	REJECT	100	4.2.1: unification/substitution failure
big-unifier.mm	ACCEPT	341	Valid database per spec
miu.mm	ACCEPT	131	Valid database per spec
peano-fixed.mm	ACCEPT	859	Valid database per spec

References

- [1] N. Megill and D. A. Wheeler, *Metamath: A Computer Language for Mathematical Proofs*, Lulu Press, 2019. Available at <http://us.metamath.org/downloads/metamath.pdf>.
- [2] M. Carneiro, “Metamath Zero: Designing a Theorem Prover Prover,” *arXiv preprint arXiv:1910.10703*, 2019.
- [3] L. de Moura and S. Ullrich, “The Lean 4 Theorem Prover and Programming Language,” in *CADE-28*, LNCS vol. 12699, Springer, 2021, pp. 625–635.
- [4] Lean Community, “Batteries: The Lean 4 Standard Library Extension,” <https://github.com/leanprover-community/batteries>, 2024.